

Lecture 4: Application Layer Protocol Ep.1 : Concept and HTTP

R wiangsripanawan



Recap: last lecture



Why IP Address?



IPv4 address Component : Network and Host



What are subnet mask, network address and broadcast address ?



Calculate subnet mask, network address and broadcast address, number of hosts and IP addresses available?



Classless Domain (a.b.c.d/n)



DHCP



Subnetting : Arranging subnets in Organization



IPv4 Problems Private IP and NAT



IPv6

Outline

Recap Last Lecture

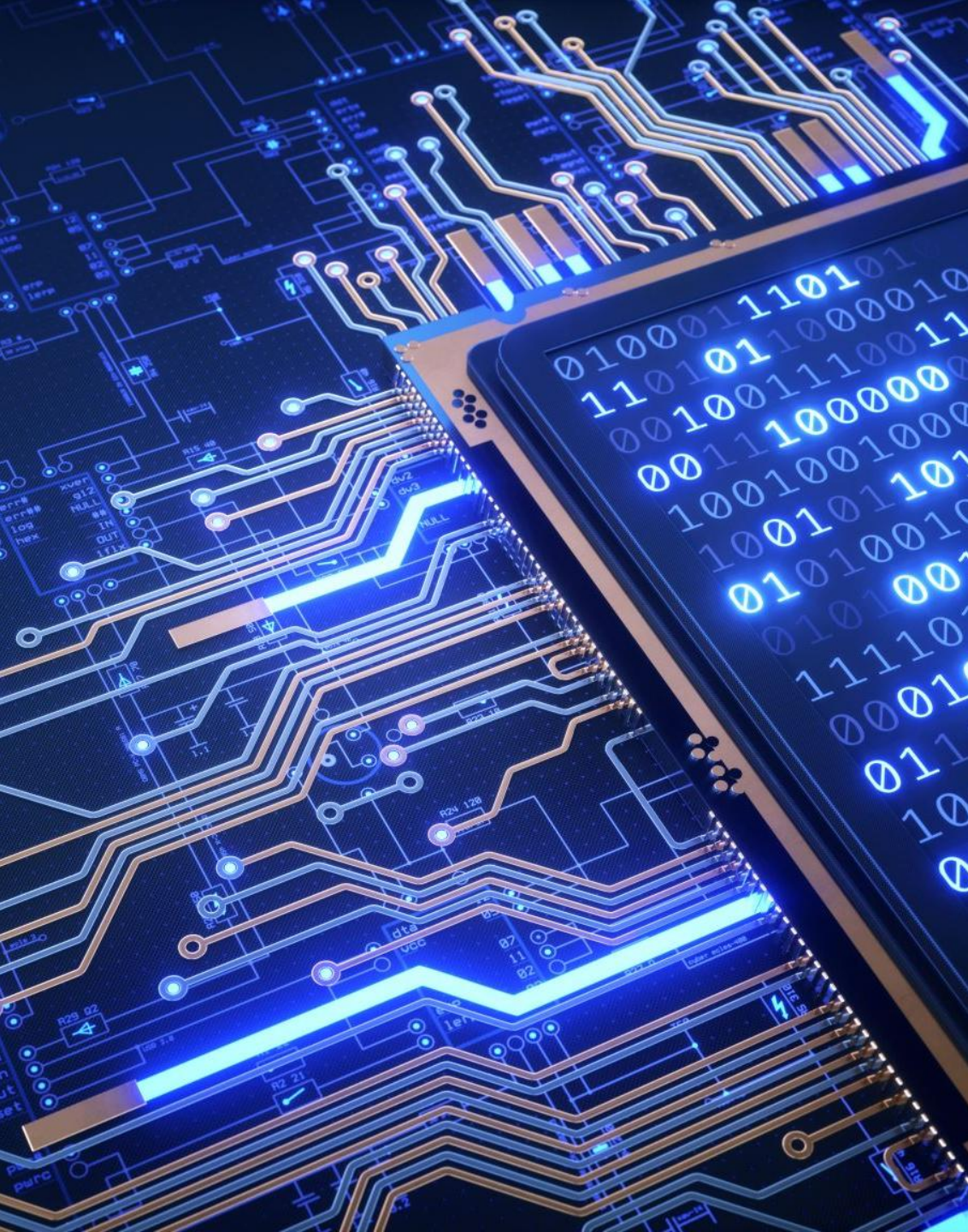
- A. Review exercises
- B. DHCP Exercises

Application Protocol Concept

HTTP

- Message Type
 - HTTP Request and HTTP Response
- Message Format
- Persistent-Non persistent
- Cookies
- Web cache

Socket API: programming network applications

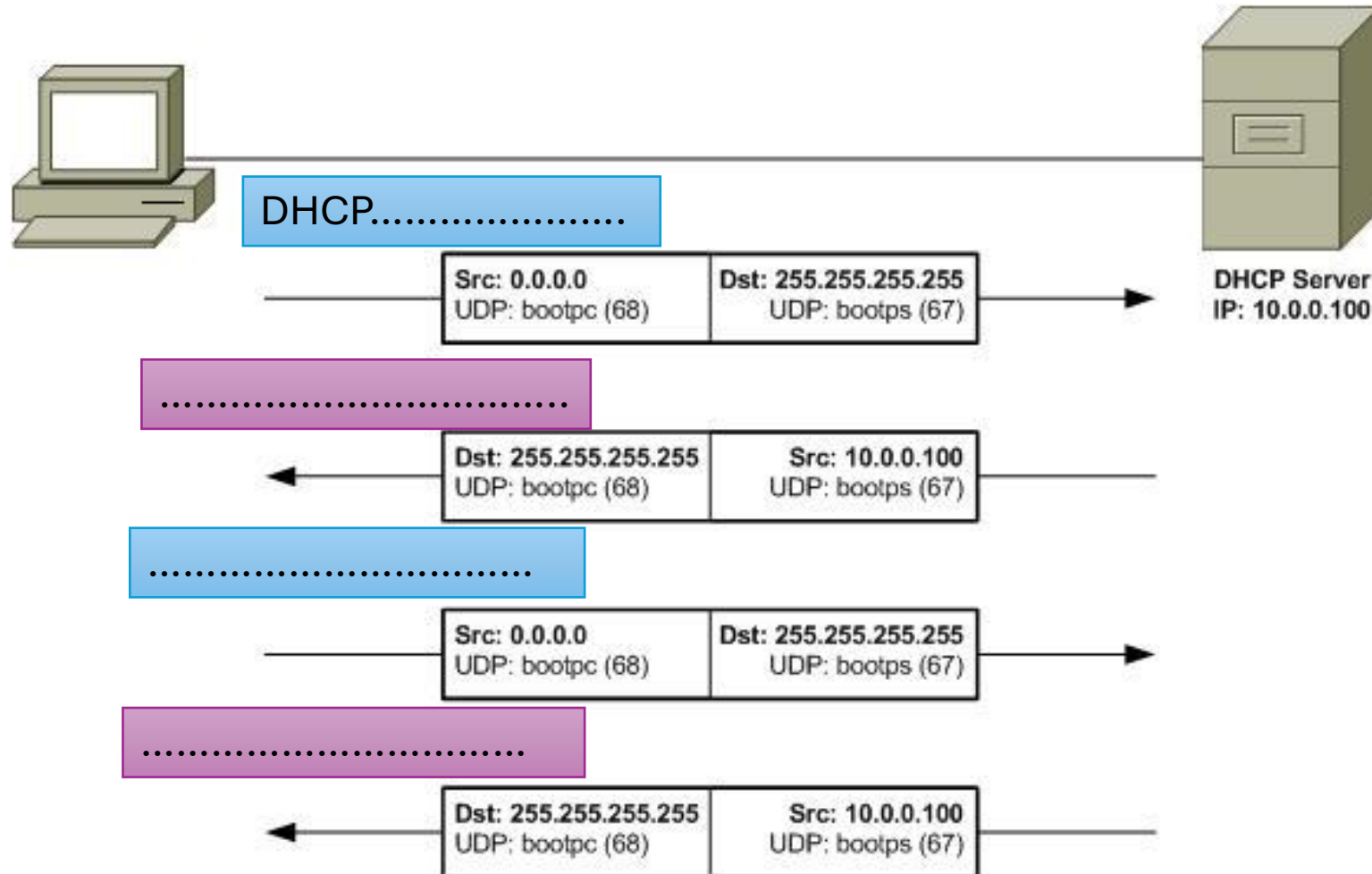


Outcome

- Understand Application Protocol concept
- Understand Client-Server and P2P Paradigm
- Understand Transports services that an application needs and be able to choose the right one for the application
- Understand HTTP Request and HTTP Response components
- Understand HTTP
 - Persistent and Nonpersistent
 - State – Cookies
 - Webcache
 - HTTP/2
- Understand socket and socket programmings

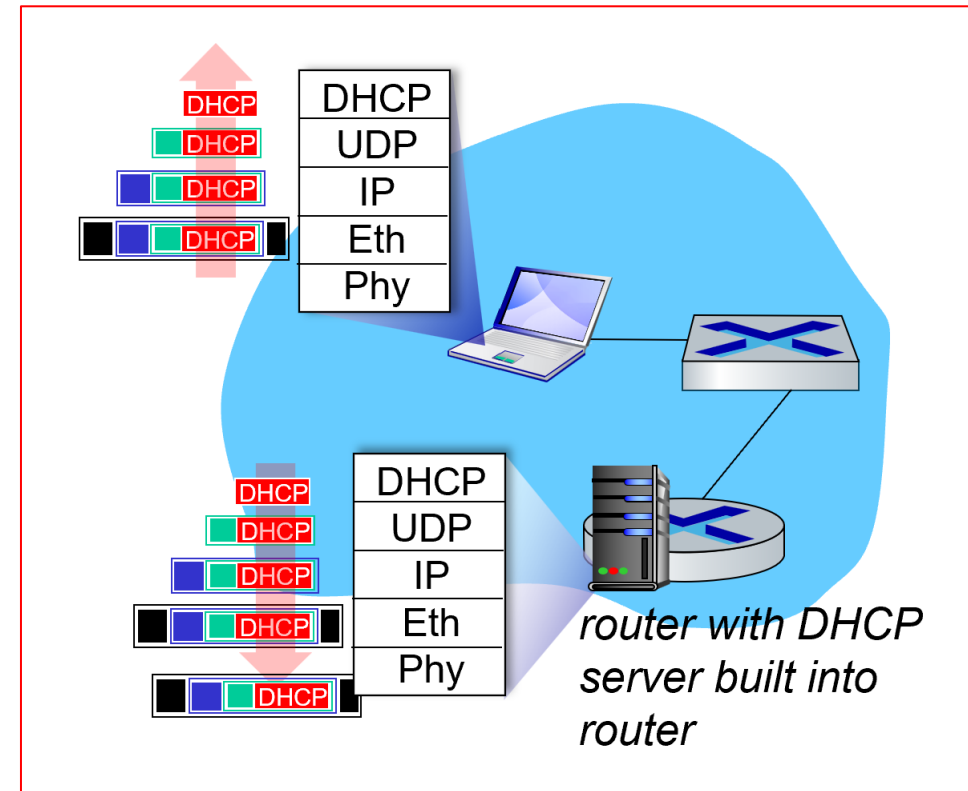
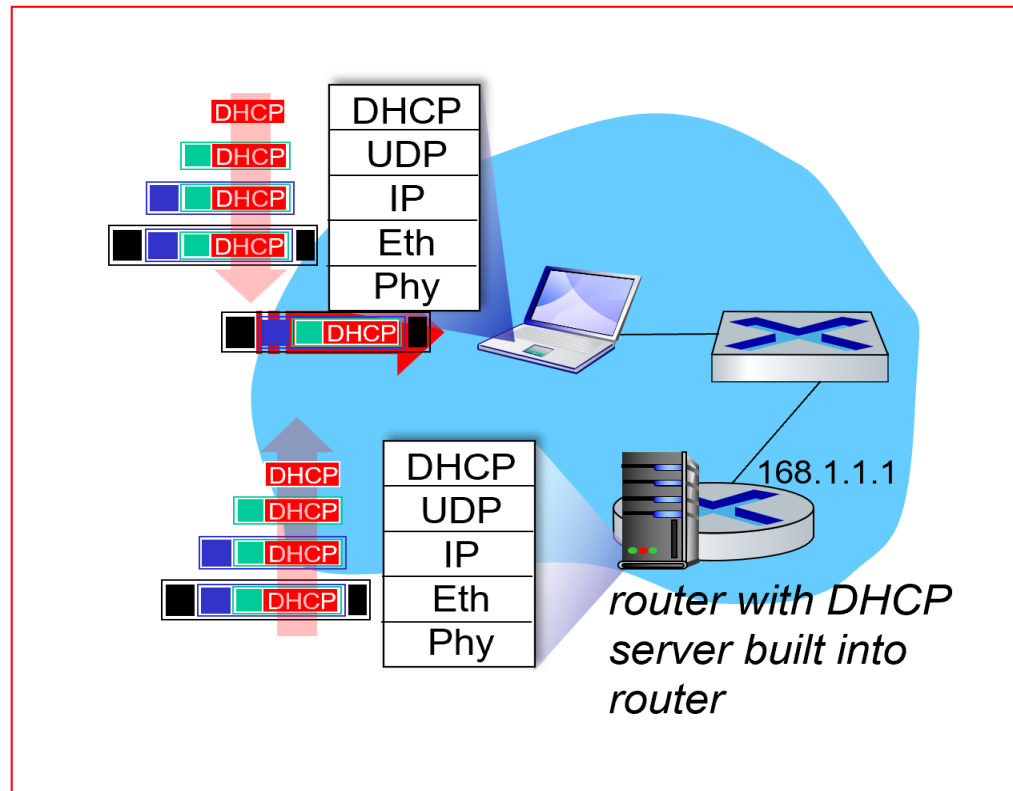
Exercise 1: Recall DHCP Messages

4.1 What are the 4 messages : answer in order.



Exercise 2: Recall DHCP Messages

2.1 From DHCP 4 messages : Which ones are in each direction?



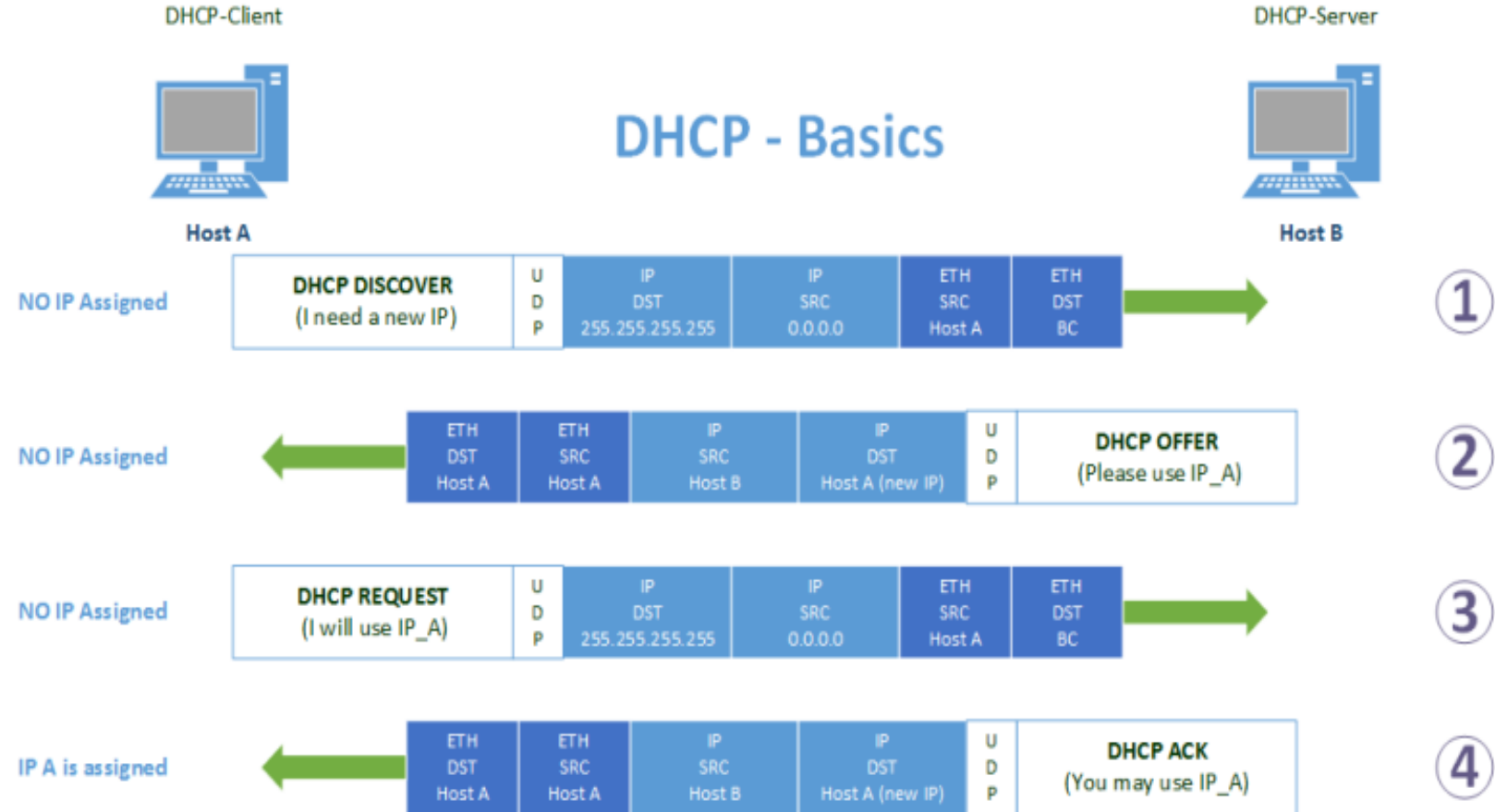
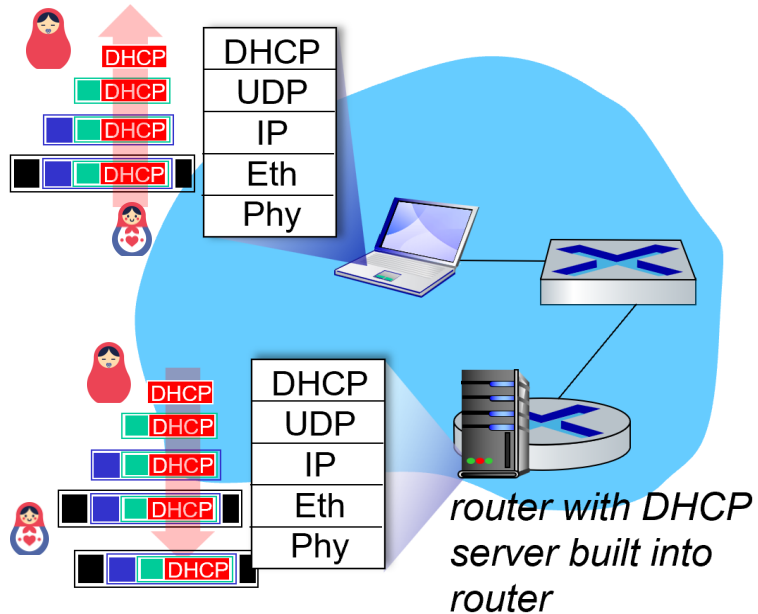
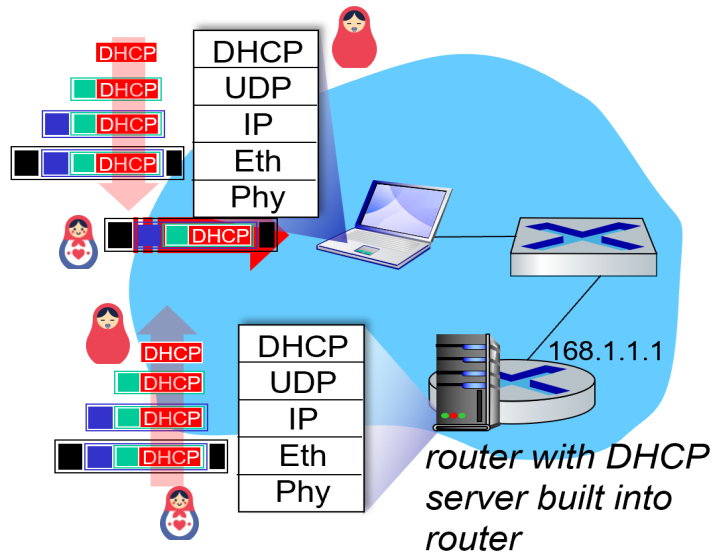
The DHCP messages in this direction can be:

-
-

The DHCP messages in this direction can be

-
-

Exercise 3.1: Headers of each protocol layer in brief

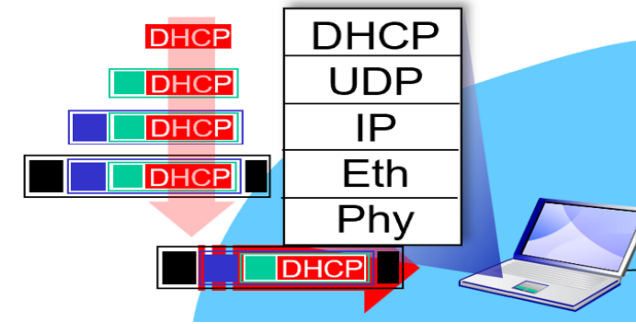


Header of Transport layer:

Header of Network layer :

Header of Data link layer:

Exercise 3.2: Headers of each protocol layer



```
> Frame 12: 392 bytes on wire (3136 bits), 392 bytes captured (3136 bits) on interface 0
> Ethernet II, Src: Centrale_08:0c:bb:80 (d0:76:58:0c:bb:80), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
> User Datagram Protocol, Src Port: 68, Dst Port: 67
```

```
✓ Bootstrap Protocol (Discover)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x2a4d077a
  Seconds elapsed: 0
  > Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: Centrale_08:0c:bb:80 (d0:76:58:0c:bb:80)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
```

```
> Option: (53) DHCP Message Type (Discover)
> Option: (61) Client identifier
> Option: (57) Maximum DHCP Message Size
> Option: (60) Vendor class identifier
> Option: (12) Host Name
> Option: (55) Parameter Request List
> Option: (82) Agent Information Option
> Option: (255) End
```

- รูปนี้แสดง DHCP message รวมทั้ง Header ใน layer ที่ Capture ได้ ให้นักศึกษาระบุว่า ส่วนไหนเป็น
 - a) DHCP Message (วงเล็บค่ะ)
 - b) Header ของ Data link layer พร้อมระบุข้อมูล
 - c) Header ของ Network layer พร้อมระบุข้อมูล
 - d) Header ของ Transport layer พร้อมระบุข้อมูล


```

> Frame 12: 392 bytes on wire (3136 bits), 392 bytes captured (3136 bits) on interface 0
> Ethernet II, Src: Centrale_08:0c:bb:80 (d0:76:58:0c:bb:80), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
> User Datagram Protocol, Src Port: 68, Dst Port: 67

```

```

✓ Bootstrap Protocol (Discover)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x2a4d077a
  Seconds elapsed: 0
  > Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: Centrale_08:0c:bb:80 (d0:76:58:0c:bb:80)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP

```

```

> Option: (53) DHCP Message Type (Discover)
> Option: (61) Client identifier
> Option: (57) Maximum DHCP Message Size
> Option: (60) Vendor class identifier
> Option: (12) Host Name
> Option: (55) Parameter Request List
> Option: (82) Agent Information Option
> Option: (255) End

```

DHCP Message Format

DHCP Message Format

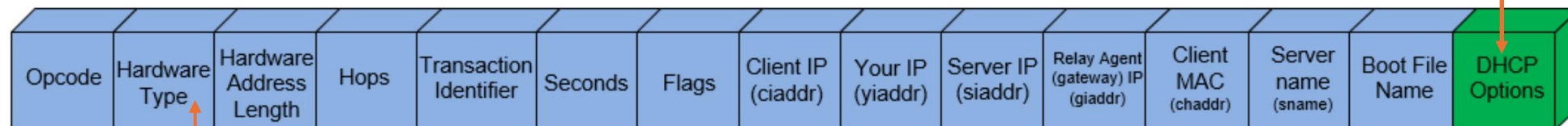
8	16	24	32
OP Code (1)	Hardware type (1)	Hardware address length (1)	Hops (1)
Transaction Identifier			
Seconds – 2 bytes		Flags – 2 bytes	
Client IP Address (CIADDR) – 4 bytes			
Your IP Address (YIADDR) – 4 bytes			
Server IP Address (SIADDR) – 4 bytes			
Gateway IP Address (GIADDR) – 4 bytes			
Client Hardware Address (CHADDR) – 16 bytes			
Server name (SNAME) – 64 bytes			
Filename – 128 bytes			
DHCP Options – variable			

D
H
C
P

M
e
s
s
a
g
e

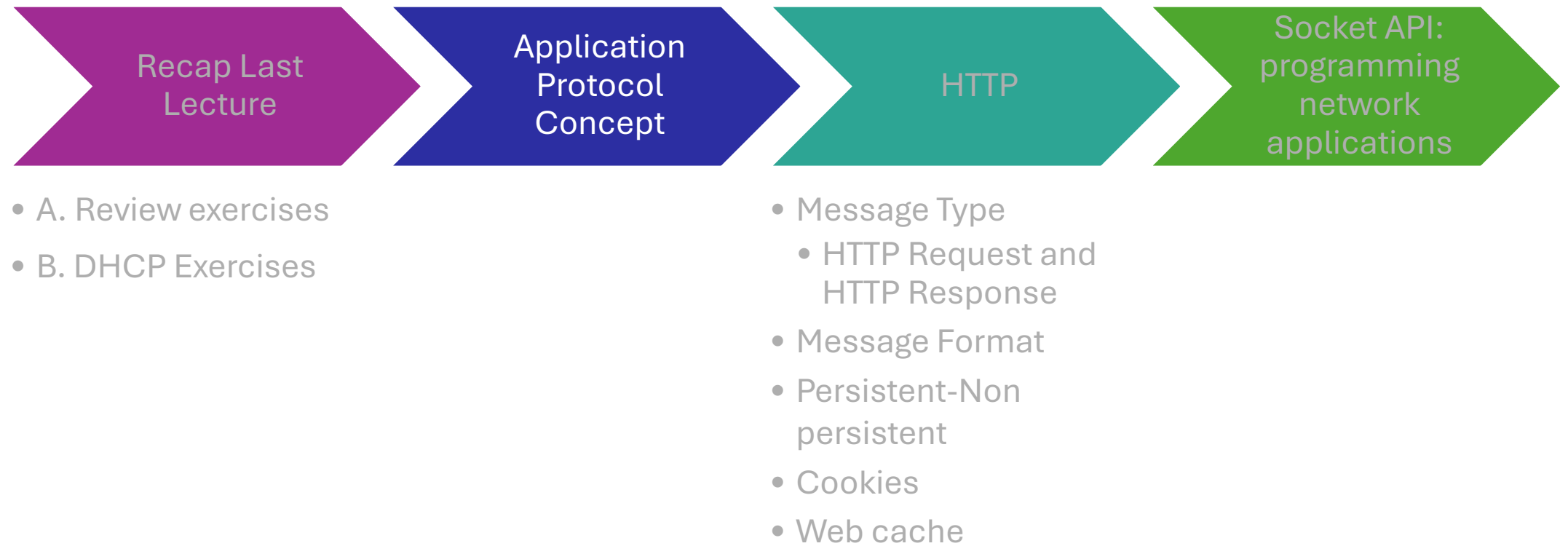
Firewall.cx

DHCP Packet



Firewall.cx

Outline



Principles of network applications



Slide page 15 - 28



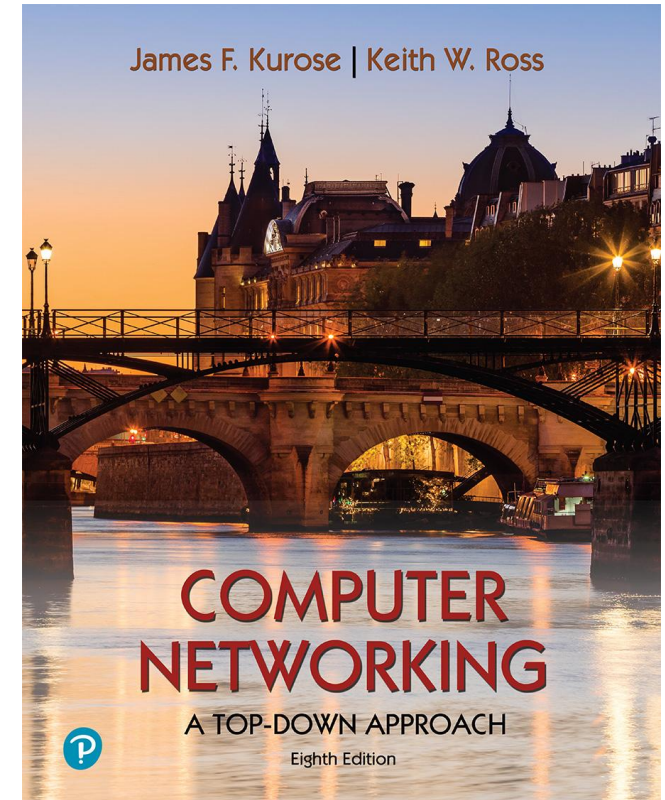
นำมาจาก **Powerpoint slides**
Chapter 2: Application Layer

Page 2-1 – 2-53 Principles of network applications



Book : Computer Networking: A Top-Down Approach 8th edition
โดย **Jim Kurose, Keith Ross,**
Pearson, 2020

https://gaia.cs.umass.edu/kurose_ross/ppt.php



Computer Networking: A Top-Down Approach

8th edition

Jim Kurose, Keith Ross
Pearson, 2020

Network application

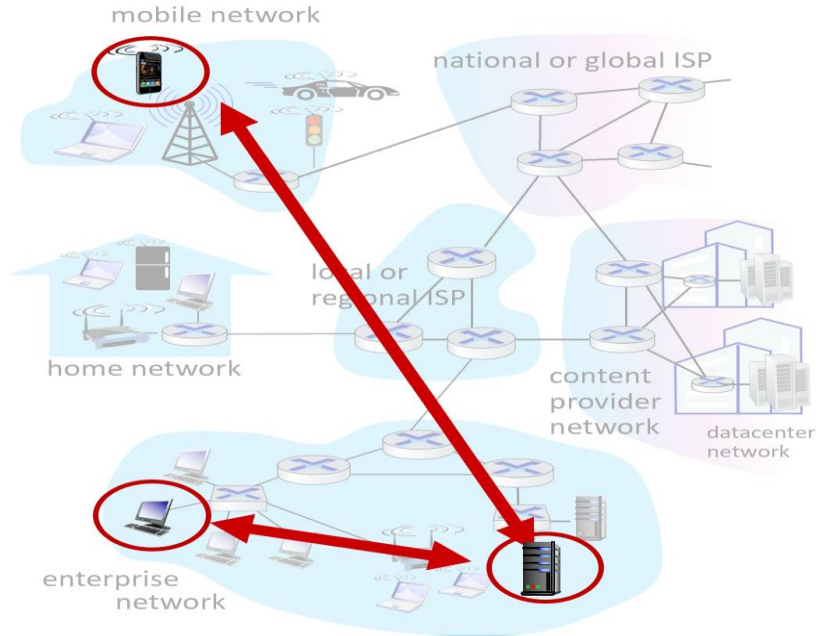
- โปรแกรมที่ทางฝั่ง **server** และ **client** คุยกันเพื่อใช้ในการทำ **Dynamic Configuration** เป็นโปรแกรมที่ใช้ โปรโตคอล **DHCP** ในการคุยกันระหว่าง **Client** และ **Server**
- โปรแกรมนี้จัดเป็นตัวอย่างหนึ่งของ **network application**
- **Exercise 4:** ให้นักศึกษายกตัวอย่าง **network application** ที่นักศึกษารู้จักมา **3 applications**

Some network apps

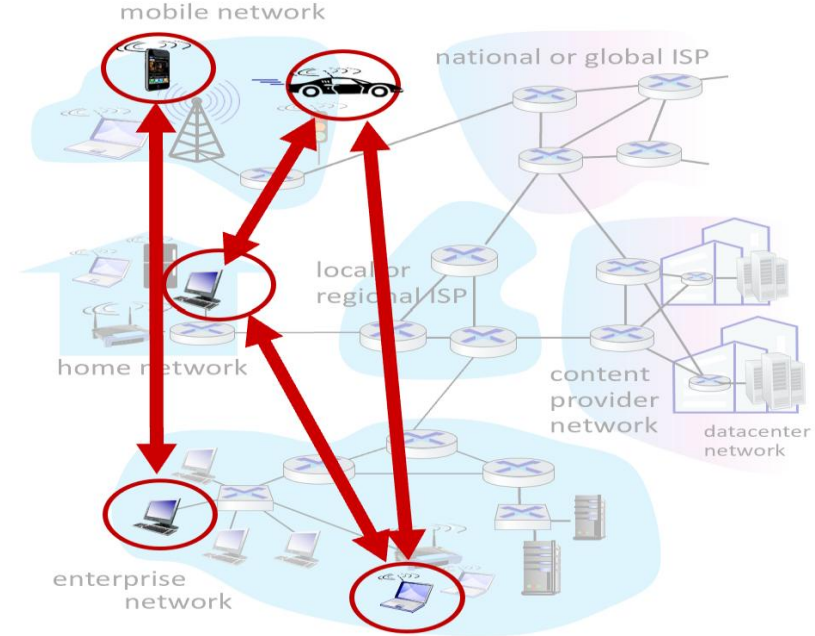
- social networking
 - Web
 - text messaging
 - e-mail
 - multi-user network games
 - streaming stored video (YouTube, Hulu, Netflix)
 - P2P file sharing
 - voice over IP (e.g., Skype)
 - real-time video conferencing (e.g., Zoom)
 - Internet search
 - remote login
 - ...
- Q: *your* favorites?

Architecture ของ Network application

มี 2 architecture



Client server architecture/ paradigm



Peer-to-Peer architecture/paradigm

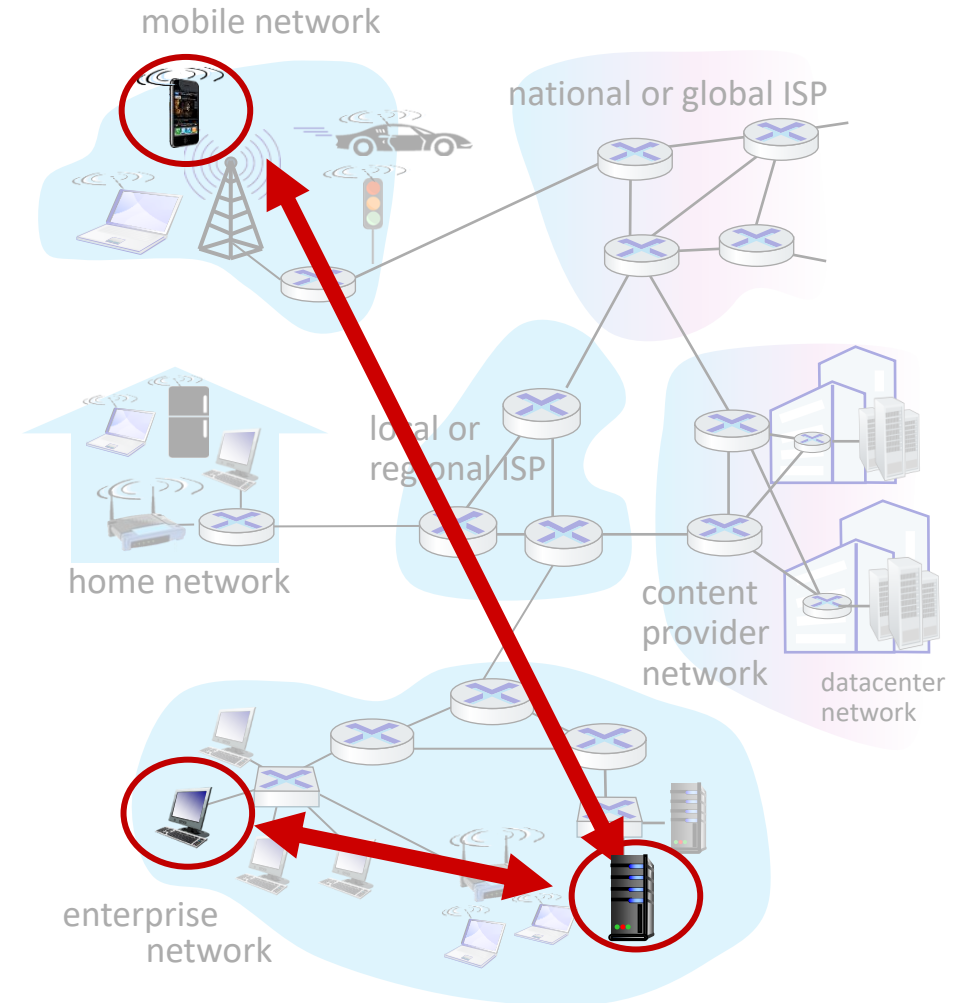
Client-server paradigm

server:

- *always-on host*
- permanent IP address
- often in data centers, for scaling

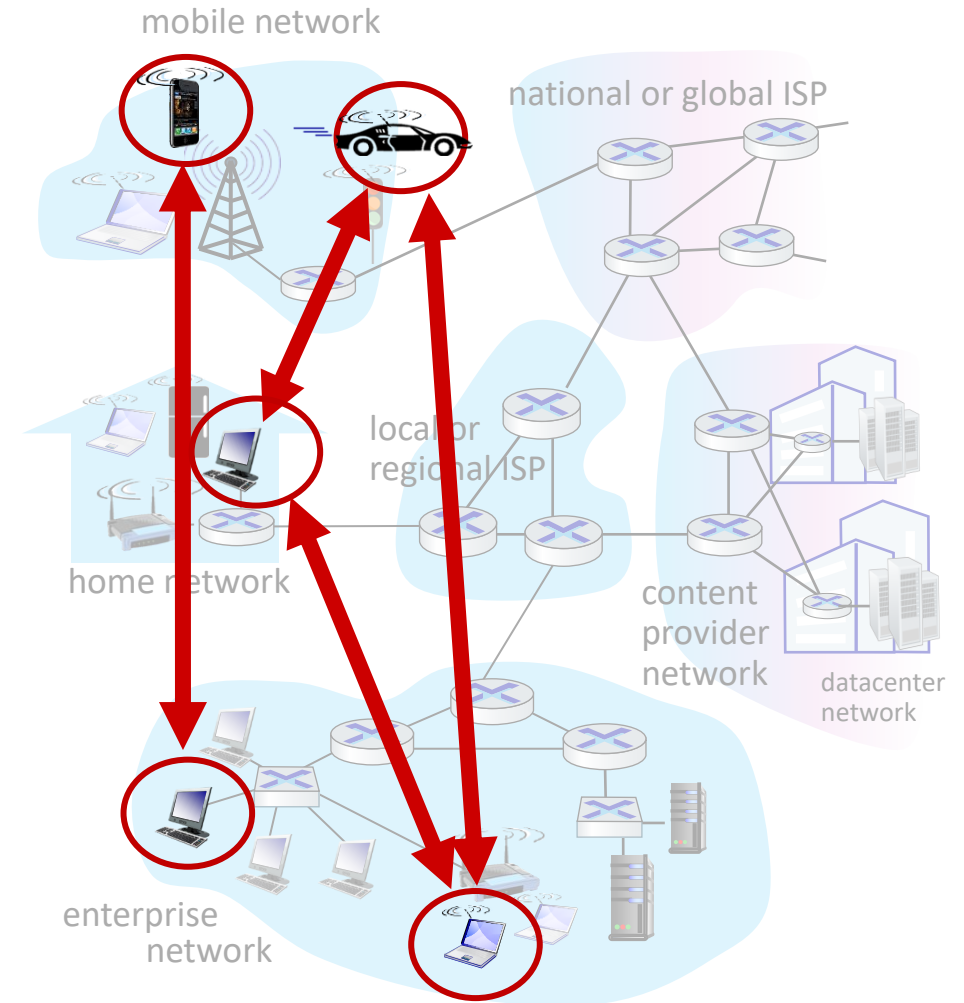
clients:

- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other
- examples: HTTP, IMAP, FTP



Peer-peer architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - *self scalability* – new peers bring new service capacity, as well as new service demands
- peers are *intermittently connected* and change IP addresses
 - complex management
- example: P2P file sharing [BitTorrent]



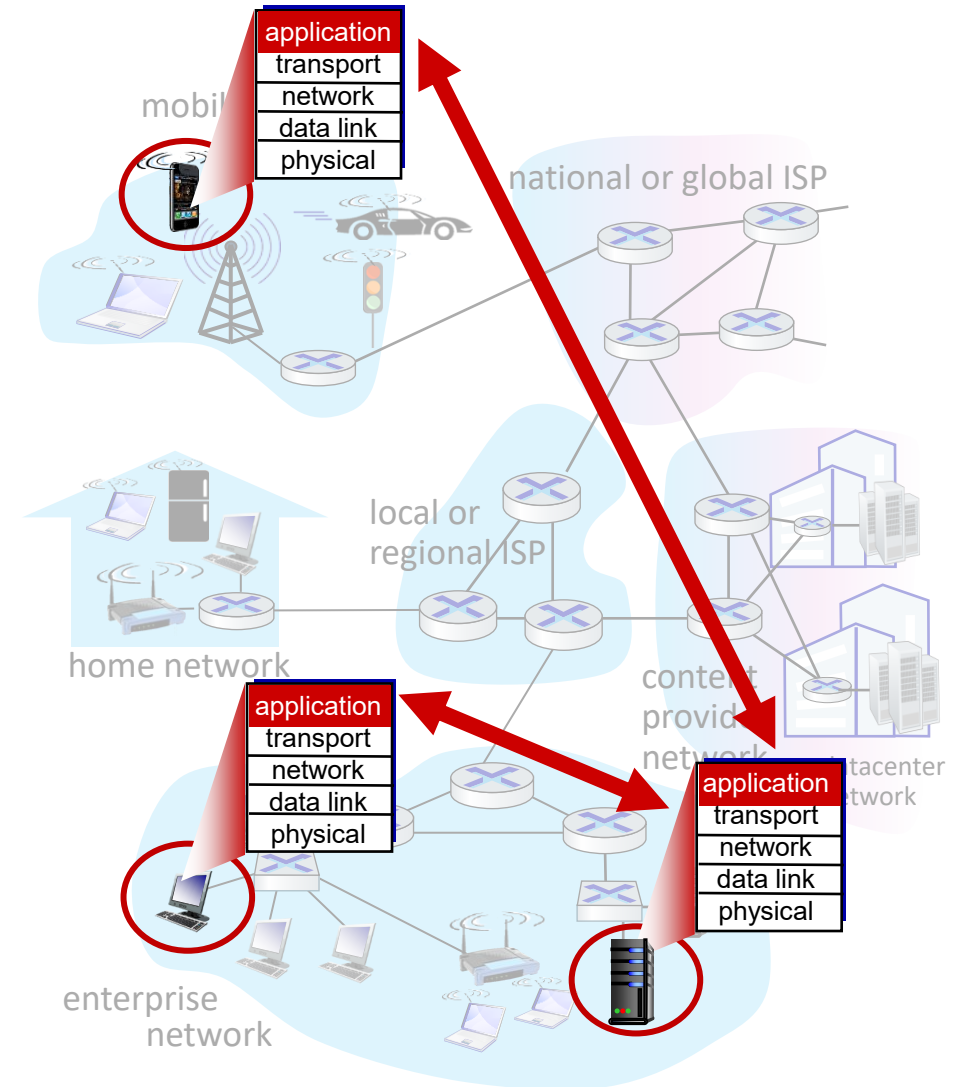
Creating a network app

write programs that:

- run on (different) end systems
- communicate over network
- e.g., web server software communicates with browser software

no need to write software for network-core devices

- network-core devices do not run user applications



Processes communicating

process: program running within a host

- within same host, two processes communicate using **inter-process communication** (defined by OS)

- ในกรณีของ network app process จะเป็น การคุยกันระหว่าง

- process ฝั่ง client และ
- process ฝั่งเซิร์ฟเวอร์
- ซึ่งสื่อสารกันผ่านทาง การแลกเปลี่ยนข้อความ
exchanging messages

clients, servers

client process: process that initiates communication

server process: process that waits to be contacted

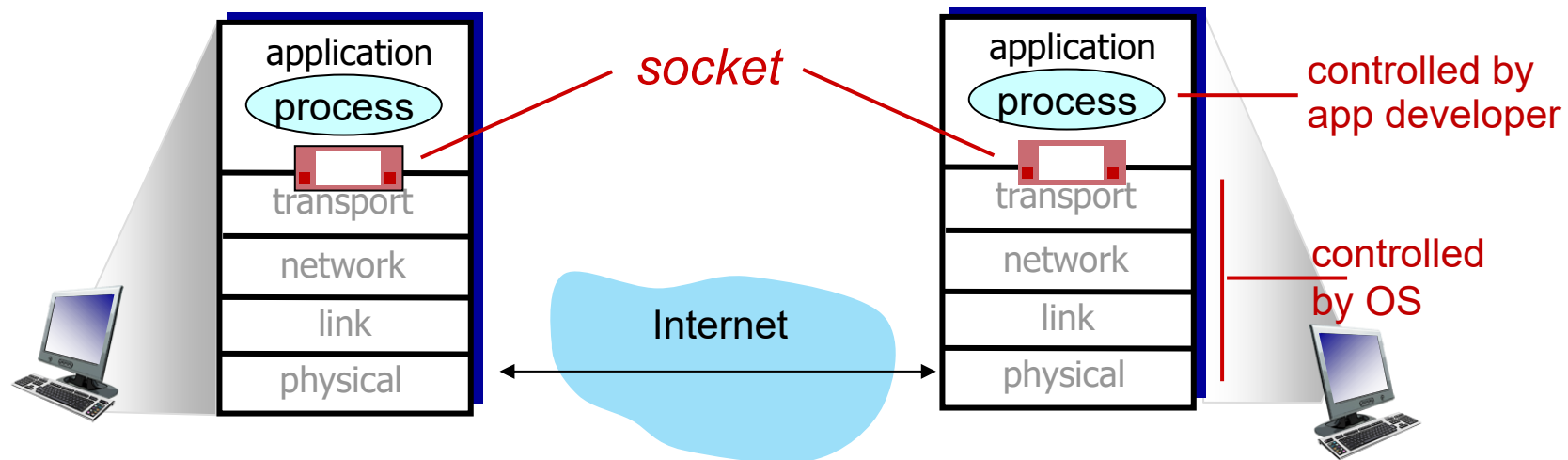
- note: applications with P2P architectures have client processes & server processes

Addressing processes (การระบุถึง client process และ server process)

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- Q: does IP address of host on which process runs suffice for identifying the process?
 - A: no, *many* processes can be running on same host
- *identifier* includes both IP address and port numbers associated with process on host.
- example port numbers:
 - HTTP server: 80
 - mail server: 25
- to send HTTP message to gaia.cs.umass.edu web server:
 - IP address: 128.119.245.12
 - port number: 80
- more shortly...

Sockets

- process sends/receives messages to/from its **socket**
- socket analogous to door
 - sending process shoves message out door
 - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
 - two sockets involved: one on each side



What is application layer?

- Protocol ที่ทำให้ Application นั้นๆ ในฝั่ง client และ Application นั้นๆ ในฝั่ง Server สื่อสารกันได้
- ฉะนั้นจึงต้องมีการกำหนด
 - types of messages exchanged
 - message syntax:
 - message semantics
 - rules for when and how processes send & respond to messages
- และ ต้องเลือก Transport layer protocol ว่าจะใช้ UDP หรือ TCP

An application-layer protocol defines:

- **types of messages exchanged**,
 - e.g., request, response
- **message syntax**:
 - what fields in messages & how fields are delineated
- **message semantics**
 - meaning of information in fields
- **rules** for when and how processes send & respond to messages

open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype, Zoom

A Request for Comments (RFC) is

- a **formal document** from the *Internet Engineering Task Force (IETF)* that
- contains specifications and organizational notes about **topics related to the internet and computer networking**, such as routing, addressing and transport technologies. – **Protocol specification**

What transport service does an app need?

data integrity

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- other apps (“elastic apps”) make use of whatever throughput they get

security

- encryption, data integrity, ...

Transport service requirements: common apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Internet transport protocols services

TCP service:

- *reliable transport* between sending and receiving process
- *flow control*: sender won't overwhelm receiver
- *congestion control*: throttle sender when network overloaded
- *connection-oriented*: setup required between client and server processes
- *does not provide*: timing, minimum throughput guarantee, security

UDP service:

- *unreliable data transfer* between sending and receiving process
- *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

Q: why bother? *Why* is there a UDP?

Internet applications, and transport protocols

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP [RFC 7230, 9110]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7230], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

A Request for Comments (RFC) is a **formal document** from the *Internet Engineering Task Force (IETF)* that contains specifications and organizational notes about topics related to the internet and computer networking, such as routing, addressing and transport technologies. – **Protocol specification**

TCP UDP not secure!!!

Vanilla TCP & UDP sockets:

- no encryption
- cleartext passwords sent into socket traverse Internet in cleartext (!)

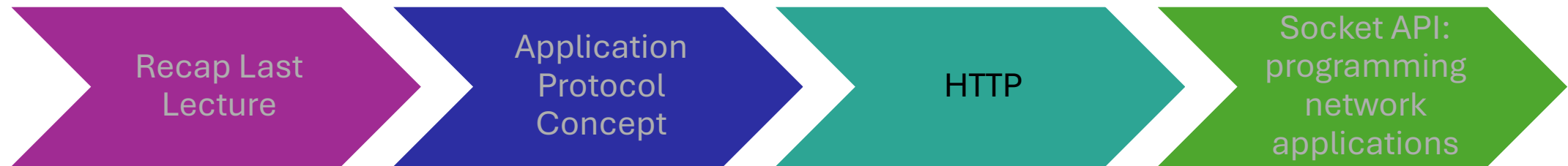


Transport Layer Security (TLS)

- provides encrypted TCP connections
- data integrity
- end-point authentication

HTTP/3.0 allow secure connection with UDP - QUIC

Outline



- A. Review exercises
- B. DHCP Exercises

- Message Type
 - HTTP Request and HTTP Response
- Message Format
- Persistent-Non persistent
- Cookies
- Web cache

Web and HTTP



Slide page



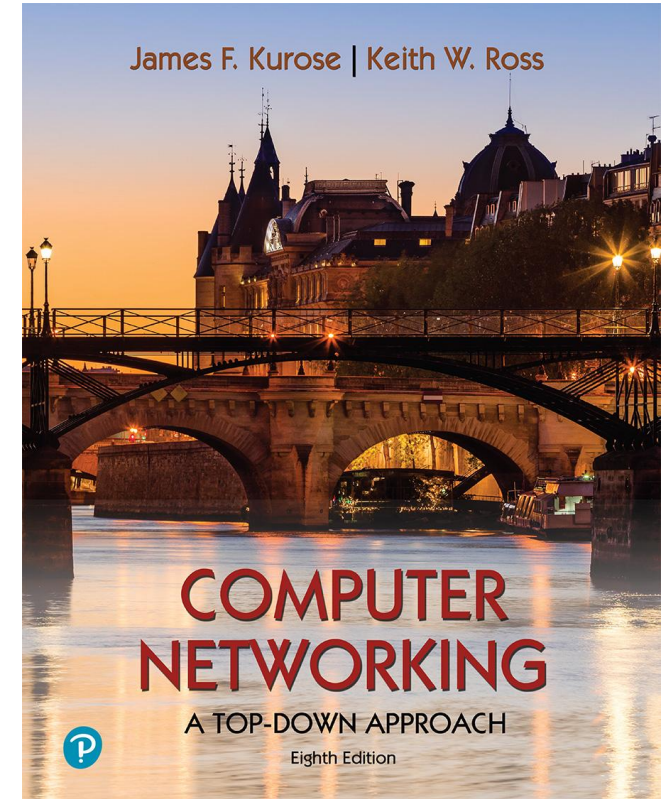
นำมาจาก **Powerpoint slides**
Chapter 2: Application Layer

Page 2-17 – 2-53
• *Web and HTTP*



**Book : Computer Networking: A
Top-Down Approach 8th edition**
โดย **Jim Kurose, Keith Ross,**
Pearson, 2020

https://gaia.cs.umass.edu/kurose_ross/ppt.php

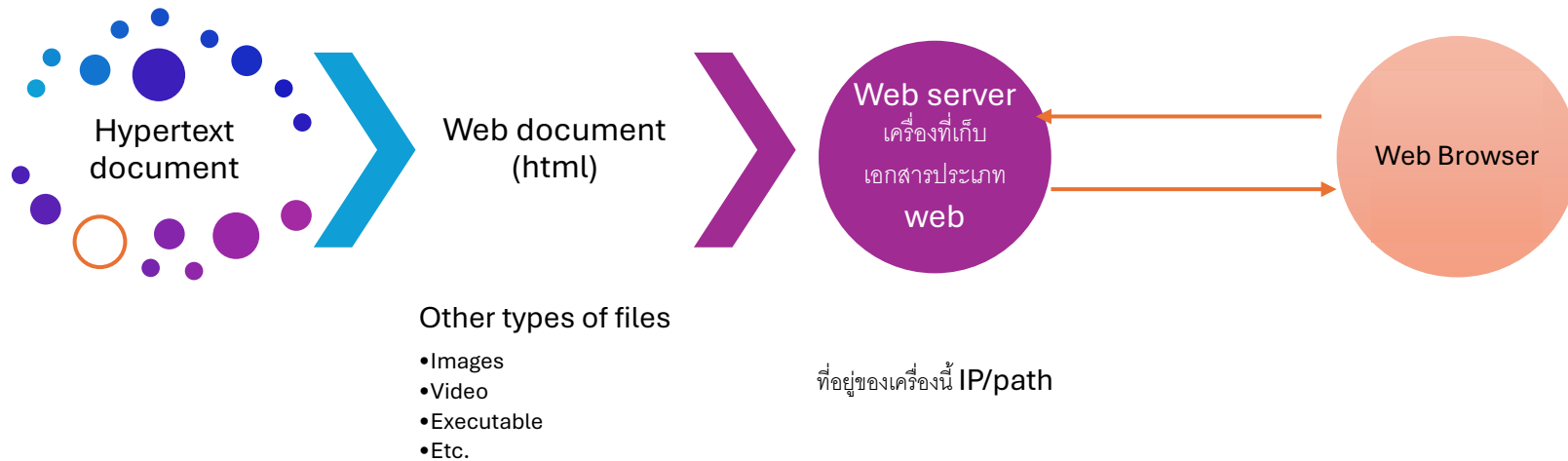


*Computer Networking: A
Top-Down Approach*

8th edition

Jim Kurose, Keith Ross
Pearson, 2020

เกร็ดความรู้เกี่ยวกับ web



Web and HTTP

First, a quick review...

- web page consists of *objects*, each of which can be stored on different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,...
- web page consists of *base HTML-file* which includes *several referenced objects, each* addressable by a *URL*, e.g.,

`www.someschool.edu/someDept/pic.gif`

host name

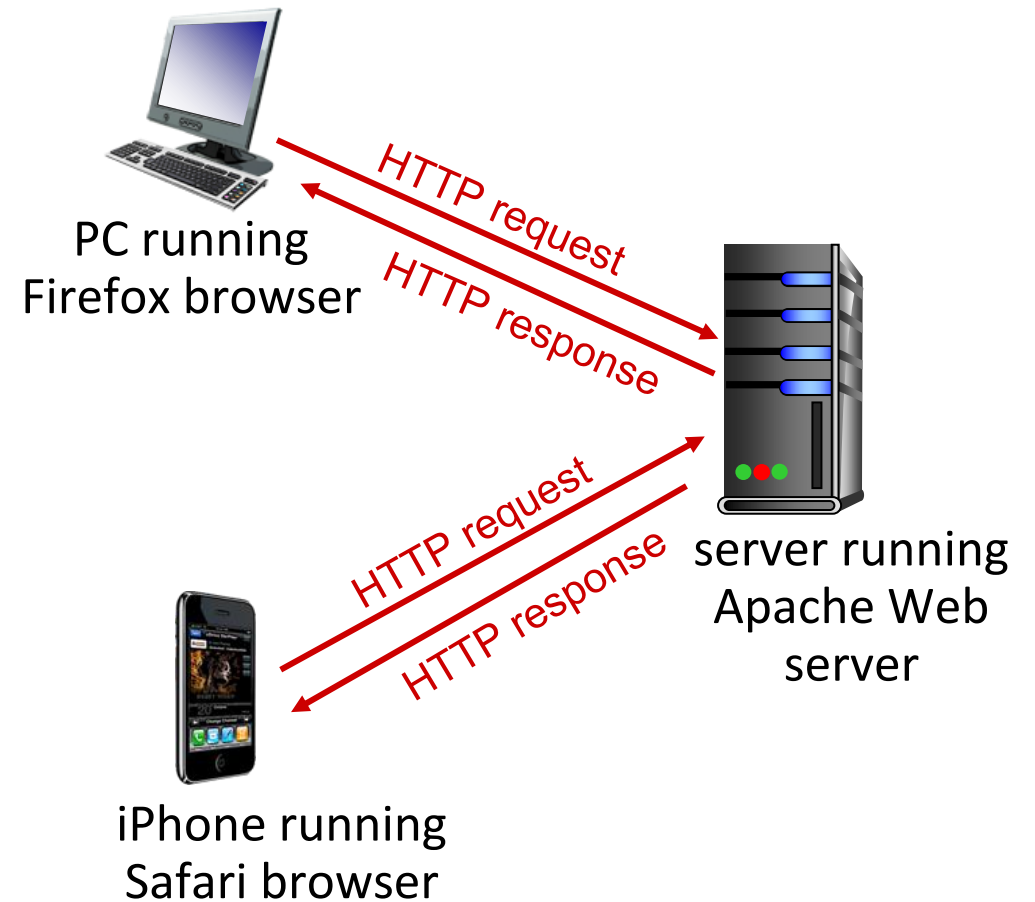
path name

HTTP overview

HTTP: hypertext transfer protocol

- Web's application-layer protocol
- client/server model:
 - *client*: browser that requests, receives, (using HTTP protocol) and “displays” Web objects
 - *server*: Web server sends (using HTTP protocol) objects in response to requests

HTTP uses TCP:



HTTP Message Type



HTTP Request



HTTP Response

- two types of HTTP messages: HTTP *request*, HTTP *response*

HTTP request message

- HTTP request message:
 - ASCII (human-readable format)

request line (GET, POST,
HEAD commands) →

header
lines

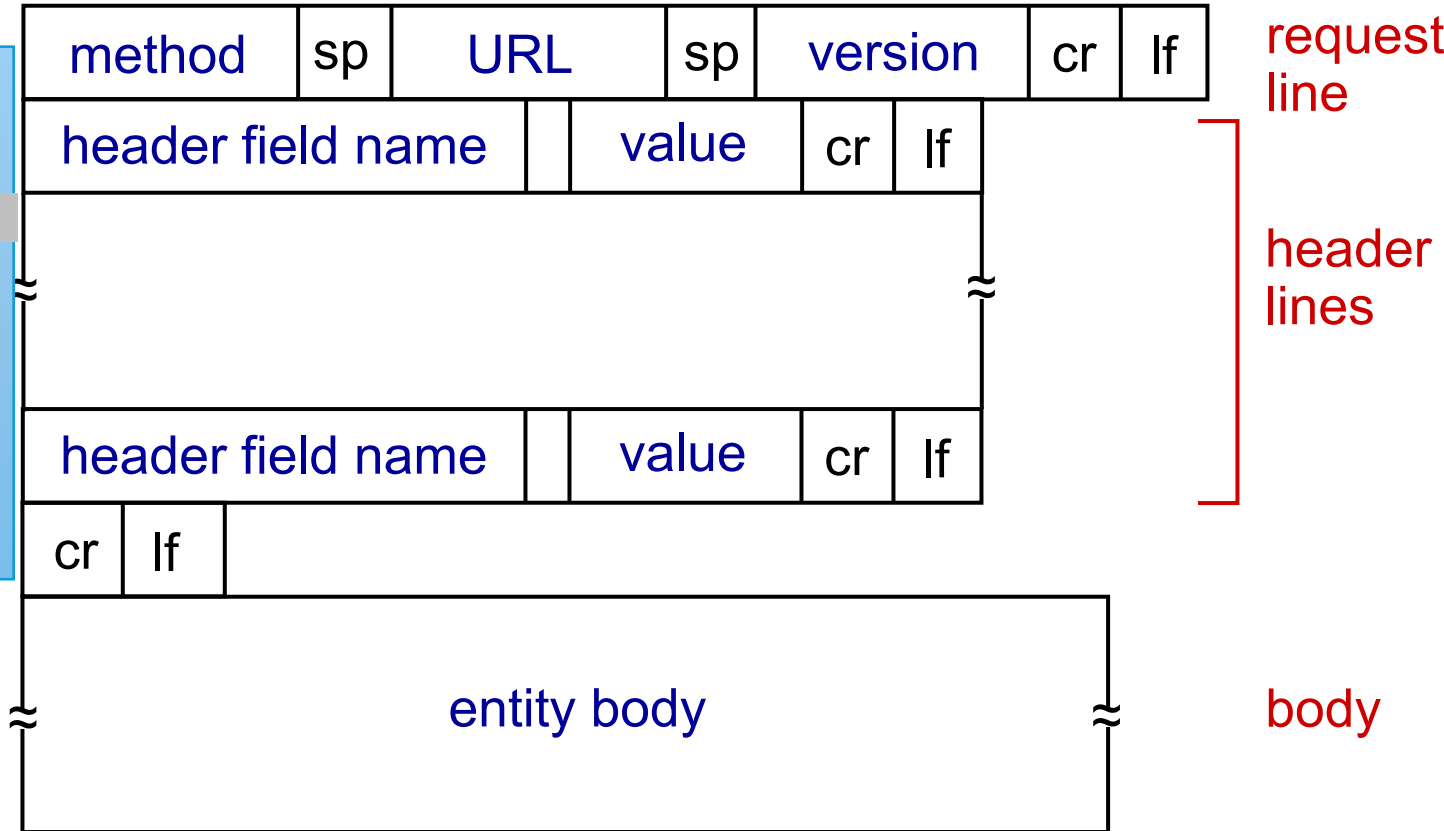
carriage return, line feed →
at start of line indicates
end of header lines

carriage return character
/ line-feed character

* Check out the online interactive exercises for more
examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

HTTP request message: general format

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel
Mac OS X 10.15; rv:80.0) Gecko/20100101
Firefox/80.0 \r\n
Accept:
text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```



Other HTTP request messages

POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):

`www.somesite.com/animalsearch?monkeys&banana`

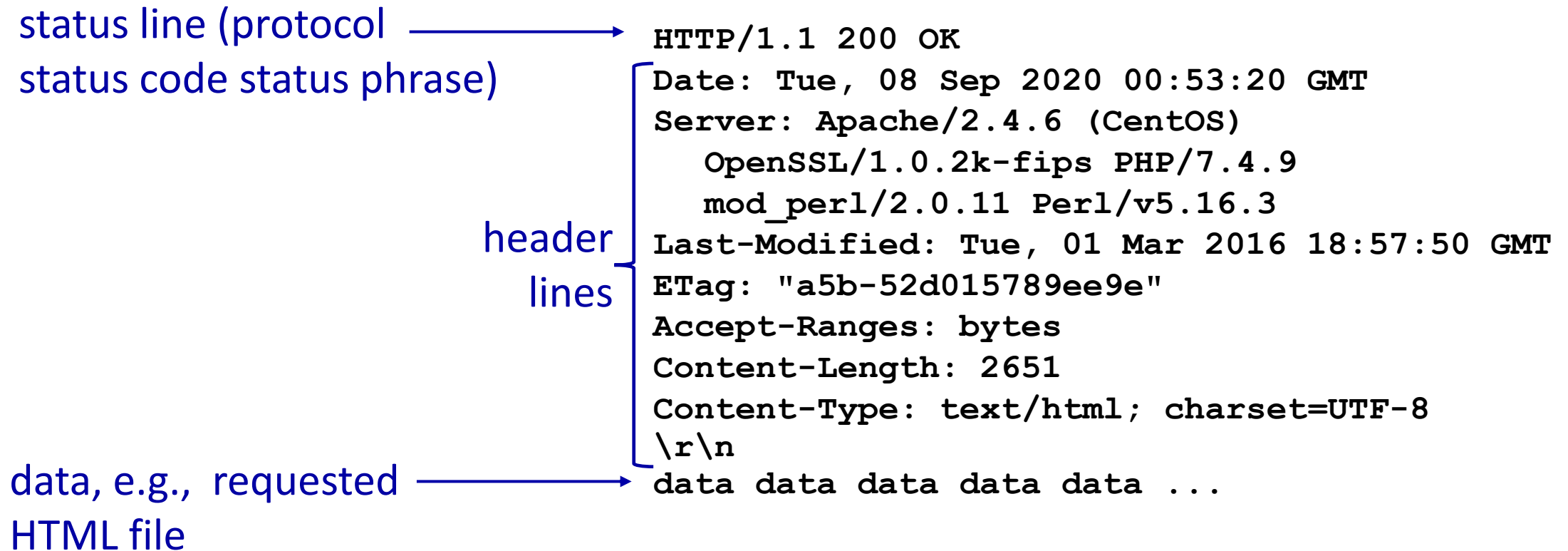
HEAD method:

- requests headers (only) that would be returned *if* specified URL were requested with an HTTP GET method.

PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

HTTP response message



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

HTTP response status codes

- status code appears in 1st line in server-to-client response message.
- some sample codes:

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

1.1xx Informational

2.2xx Success

3.3xx Redirection

4.4xx Client Error

5.5xx Server Error

สรุป

- HTTP Request

- Request Line

- *Method URL Version \r\n*

- **GET /index.html HTTP/1.1\r\n**

- Header Lines

- Body (ค่านี้อาจจะมีหรือไม่มีก็ได้)

- HTTP Response

- Status Line

- *Protocol Status-code Status phrase*

- **HTTP/1.1 200 OK**

- Header Lines

- Data

HTTP overview (a closer look)

HTTP uses TCP by:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP connections: two types

Non-persistent HTTP

1. TCP connection opened
2. at most one object sent over TCP connection
3. TCP connection closed

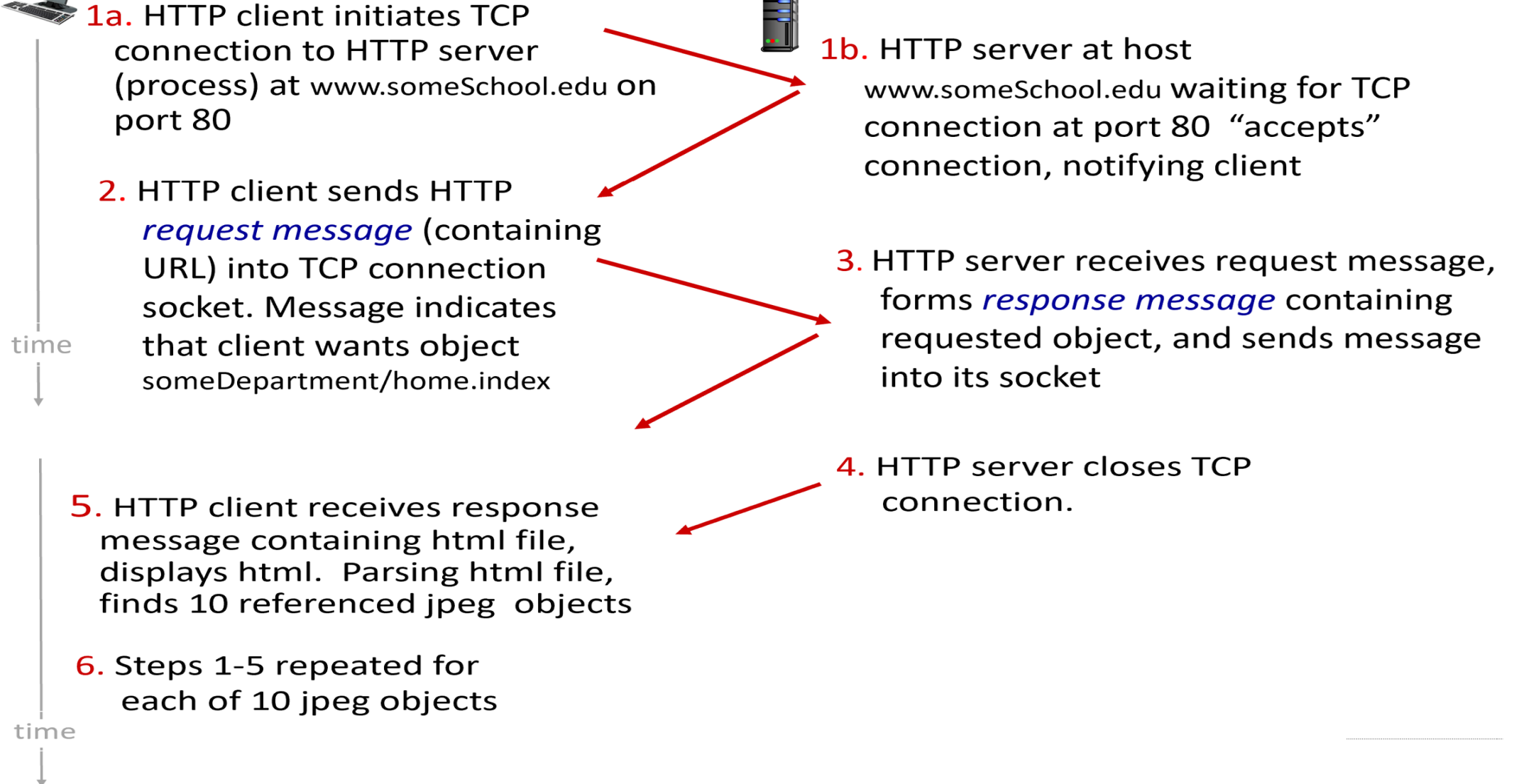
downloading multiple objects required multiple connections

Persistent HTTP

- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed

Non-persistent HTTP: example

User enters URL: `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)

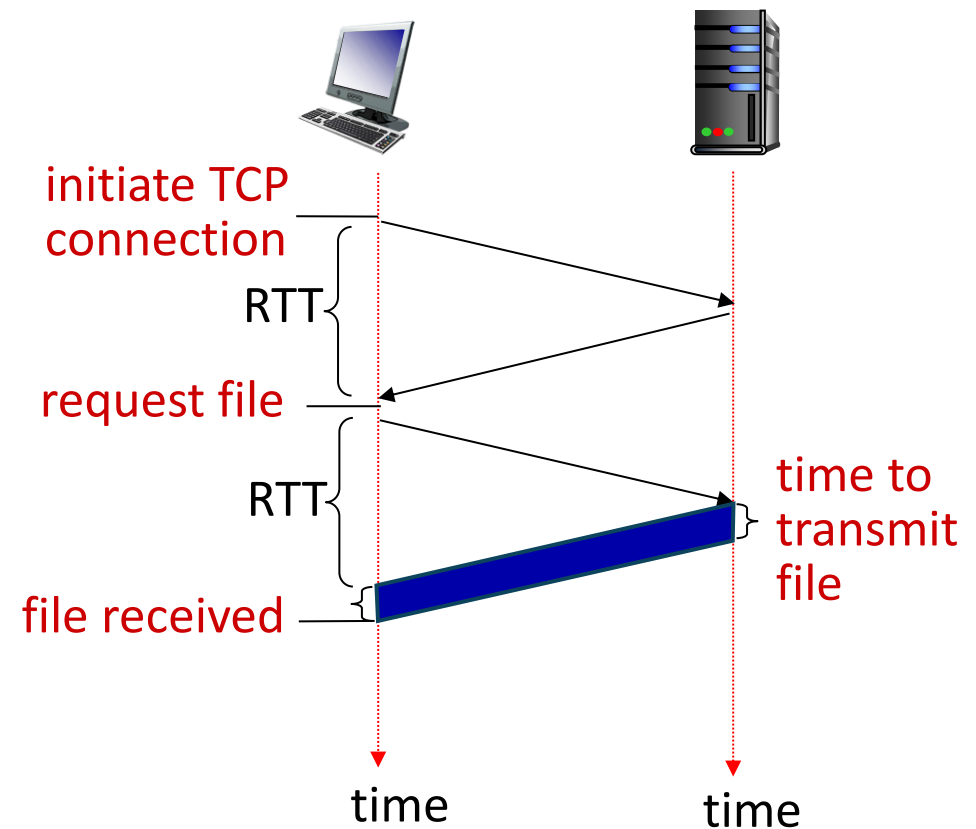


Non-persistent HTTP: response time

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time (per object):

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time



Non-persistent HTTP response time = $2RTT + \text{file transmission time}$

Exercise 5: ถ้า web page ที่นักศึกษาเข้าชมมีทั้งหมด 10 objects จงหา HTTP Response time

ให้ Transmission Time ของ Object1 object2 object3 ... object10 เป็น $t_1, t_2, t_3, \dots, t_{10}$ ตามลำดับ

- กรณี Non- Persistent HTTP

- กรณี Persistent HTTP

Persistent HTTP (HTTP 1.1)

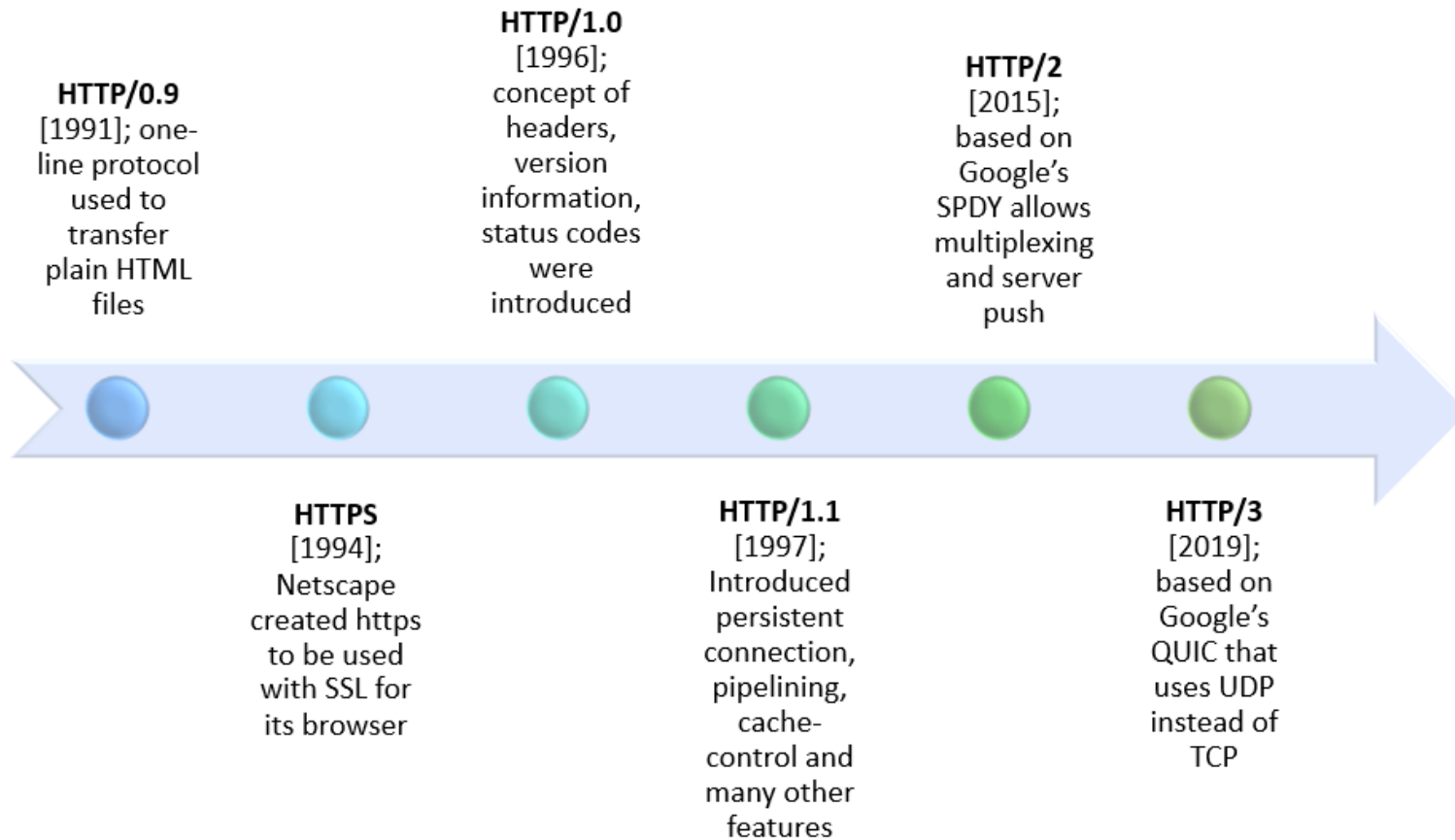
Non-persistent HTTP issues:

- requires 2 RTTs per object
- OS overhead for *each* TCP connection
- browsers often open multiple parallel TCP connections to fetch referenced objects in parallel

Persistent HTTP (HTTP1.1):

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects (cutting response time in half)

HTTP Versions



Comparison of protocol stack changes delivered with each new version after HTTP/1.0		
HTTP/1.1	HTTP/2	HTTP/3
- Some methods and response codes are added. "Keep-Alive" becomes officially supported. "Host" header becomes supported for Virtual Domain. - Syntax and semantics are separated.	- Support of parallel request transmission by "stream" (elimination of HTTP HoL Blocking). - Addition of flow-control and prioritization function in units of "stream". - Addition of server-push function (send related file without request.)	- Lower protocol changes from TCP+TLS to UDP+QUIC - Streams and flow-control function are moved to QUIC. - Parallel request transmission is supported by QUIC stream (eliminating TCP HoL Blocking).

Cookies : http ช่วงแรก เป็น stateless protocol

protocols that maintain
“state” are complex!

- past history (state) must be maintained
- if server/client crashes, their views of “state” may be inconsistent, must be reconciled

Original HTTP is “stateless”

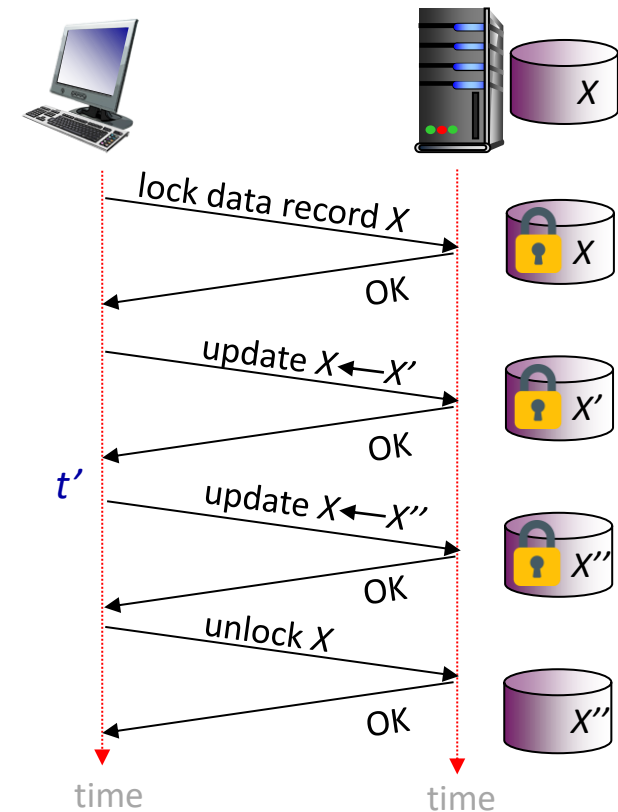
- server maintains *no* information about past client requests

Maintaining user/server state: cookies

Recall: HTTP GET/response interaction is *stateless*

- no notion of multi-step exchanges of HTTP messages to complete a Web “transaction”
 - no need for client/server to track “state” of multi-step exchange
 - all HTTP requests are independent of each other
 - no need for client/server to “recover” from a partially-completed-but-never-completely-completed transaction

a *stateful protocol*: client makes two changes to X, or none at all



Q: what happens if network connection or client crashes at t' ?

Maintaining user/server state: cookies

Web sites and client browser use *cookies* to maintain some state between transactions

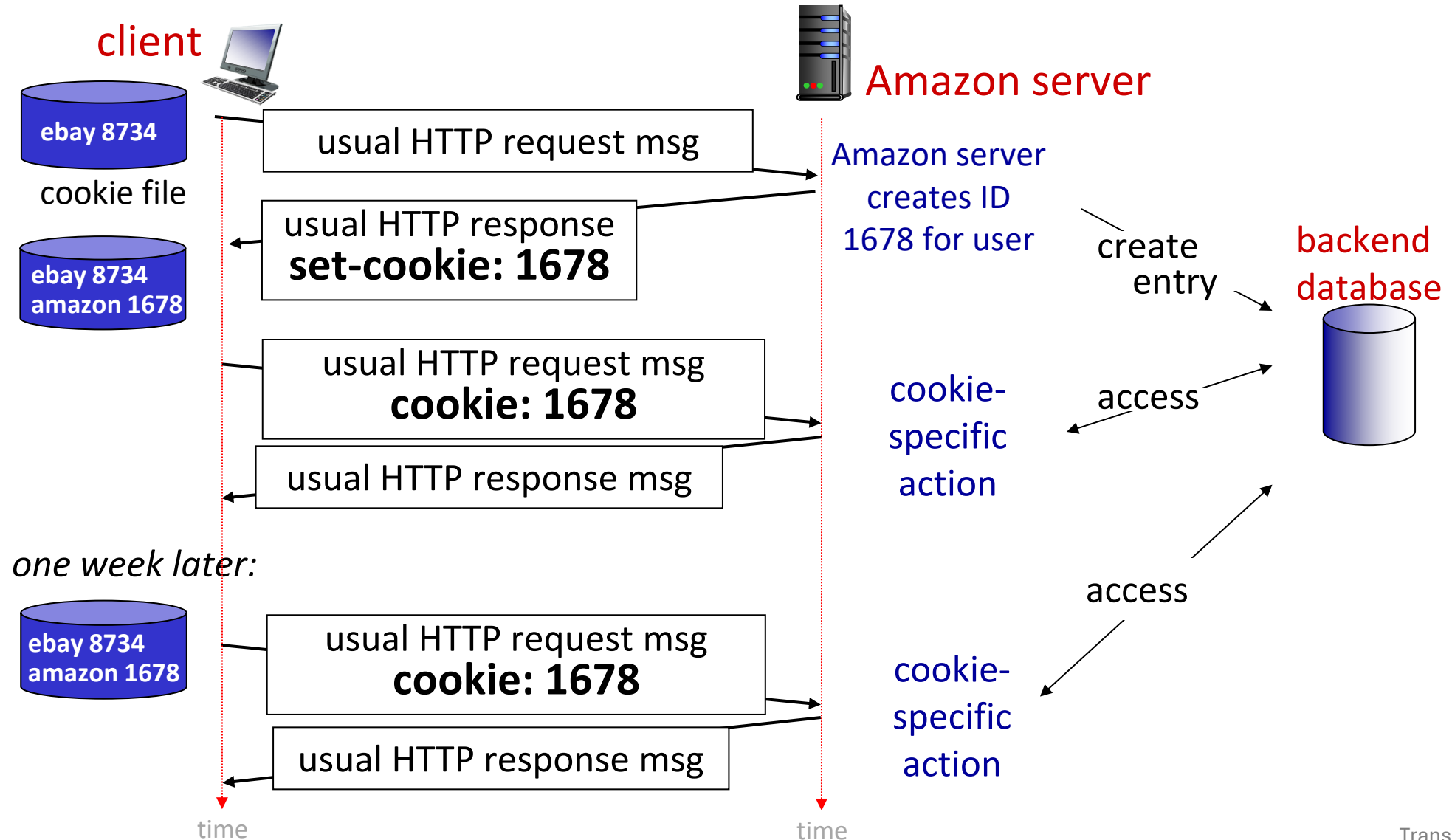
four components:

- 1) cookie header line of HTTP *response* message
- 2) cookie header line in next HTTP *request* message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web site

Example:

- Susan uses browser on laptop, visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
 - unique ID (aka “cookie”)
 - entry in backend database for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to “identify” Susan

Maintaining user/server state: cookies



HTTP cookies: comments

What cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

Challenge: How to keep state?

- *at protocol endpoints:* maintain state at sender/receiver over multiple transactions
- *in messages:* cookies in HTTP messages carry state

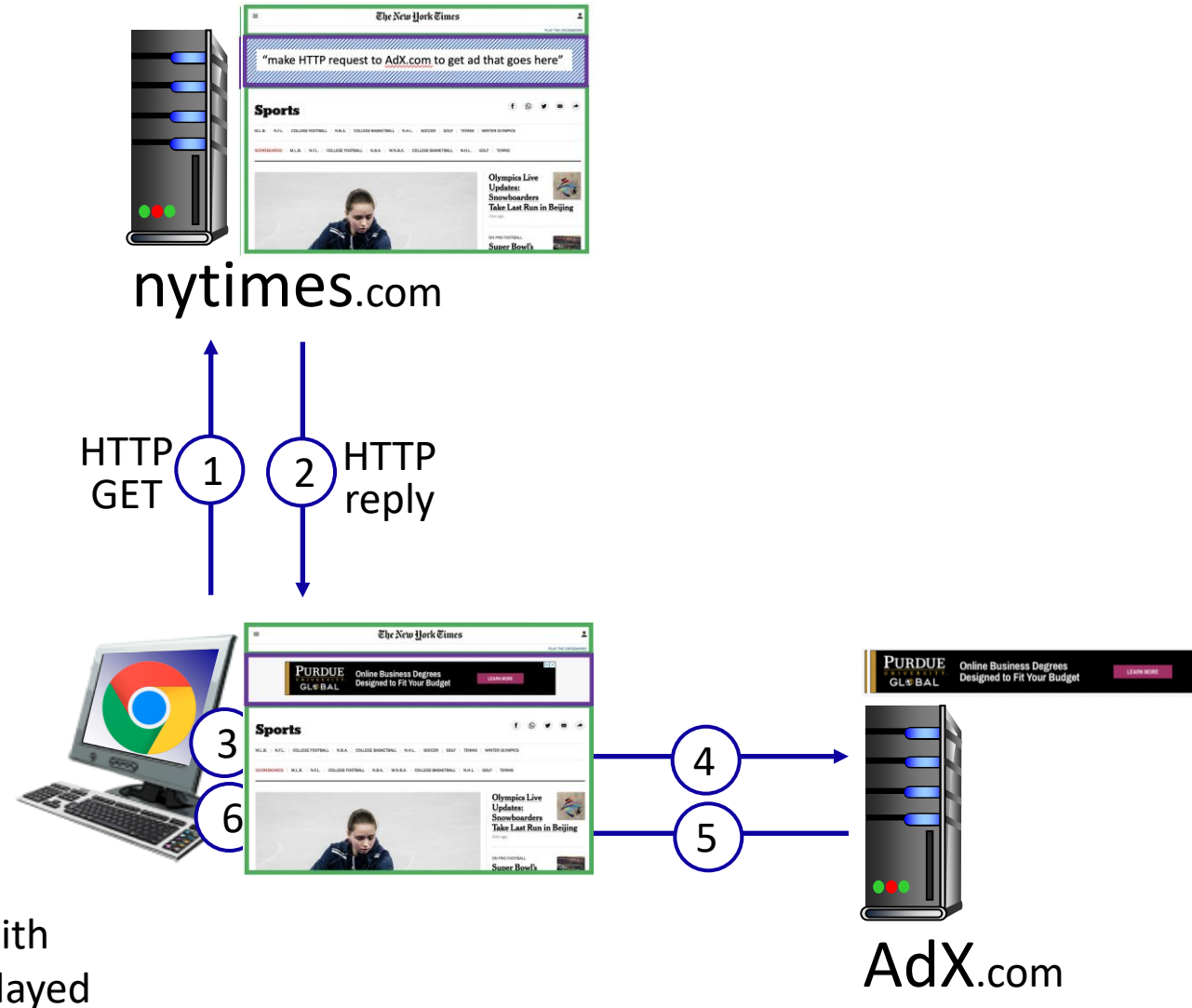
aside

cookies and privacy:

- cookies permit sites to *learn* a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

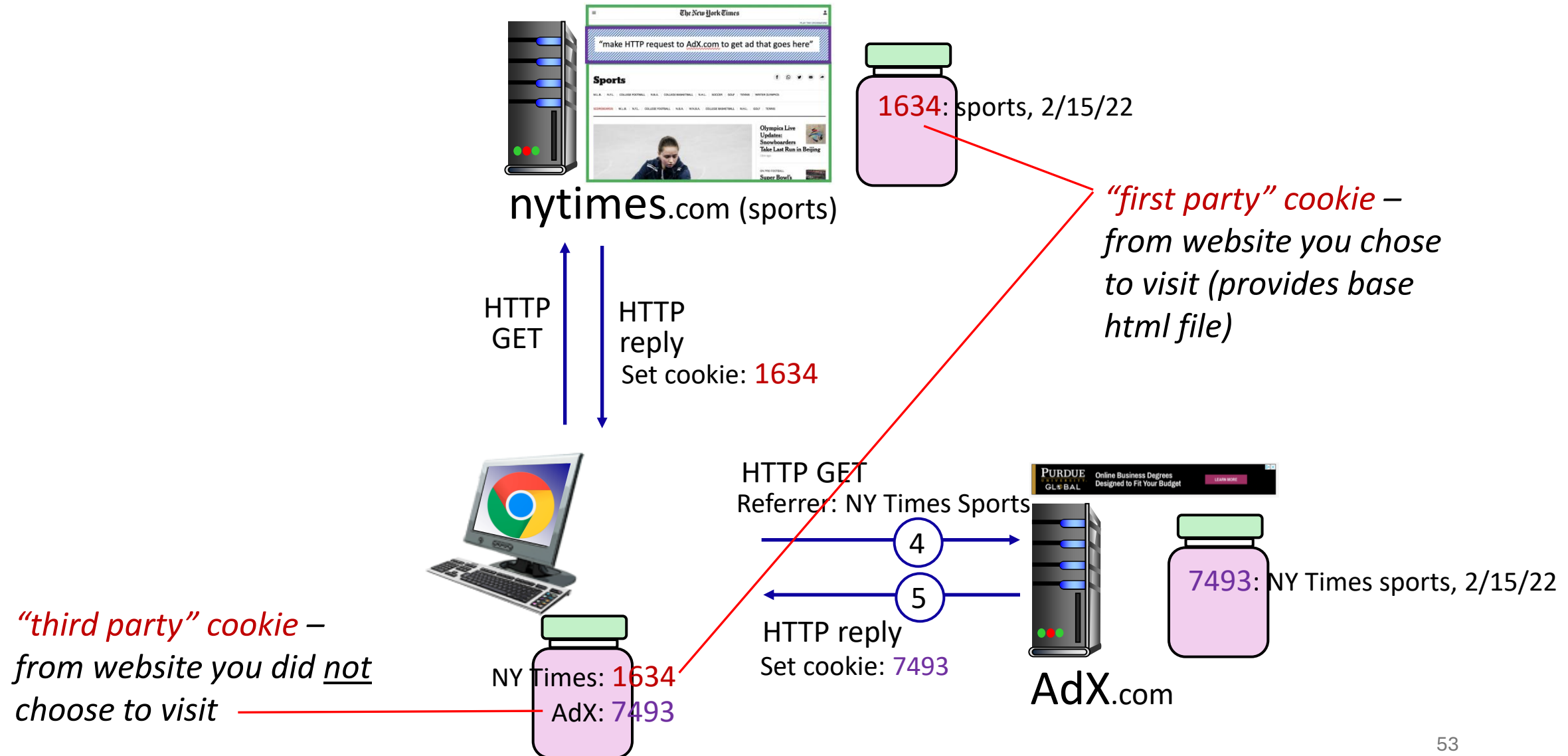
Example: displaying a NY Times web page

- 1 GET base html file
- 2 from nytimes.com
- 4 fetch ad from
- 5 AdX.com
- 7 display composed page

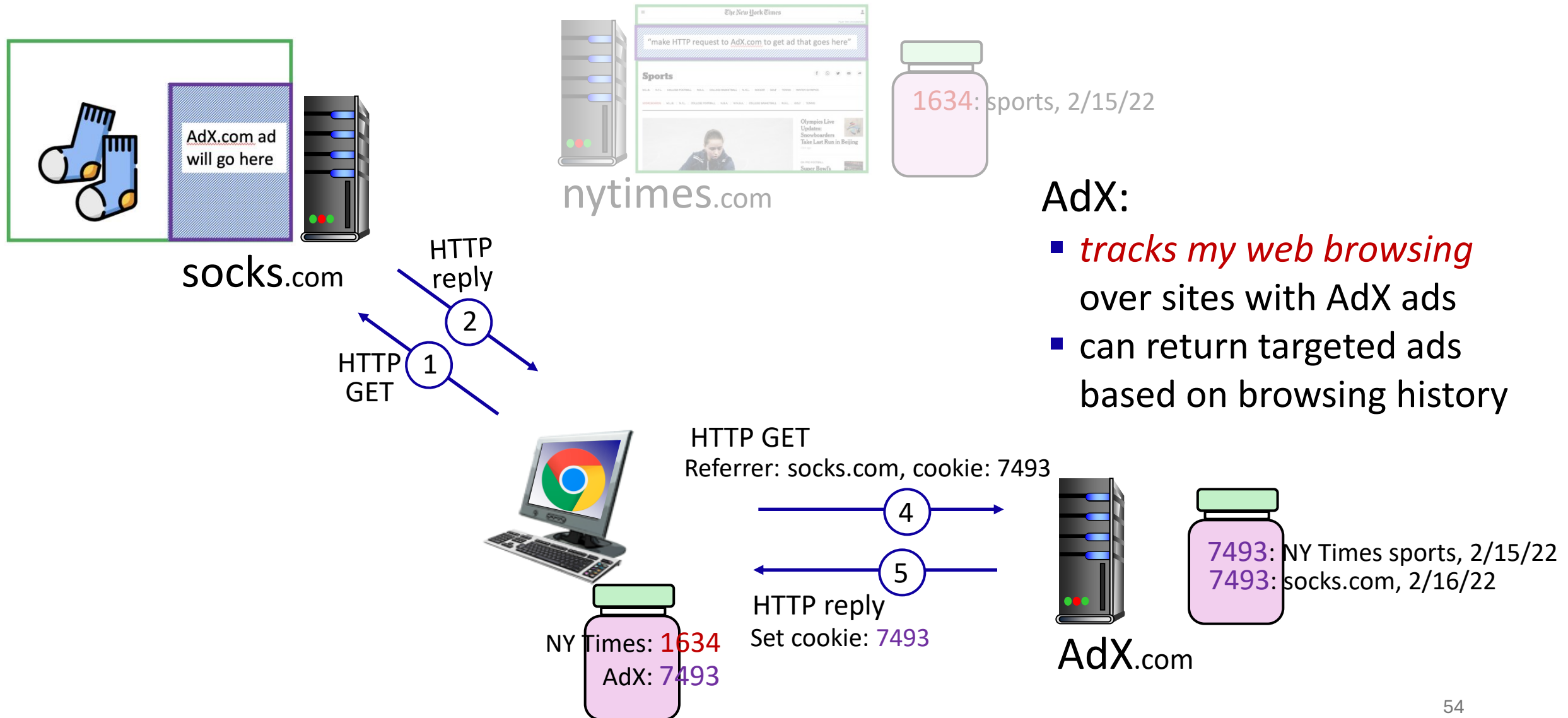


NY times page with
embedded ad displayed

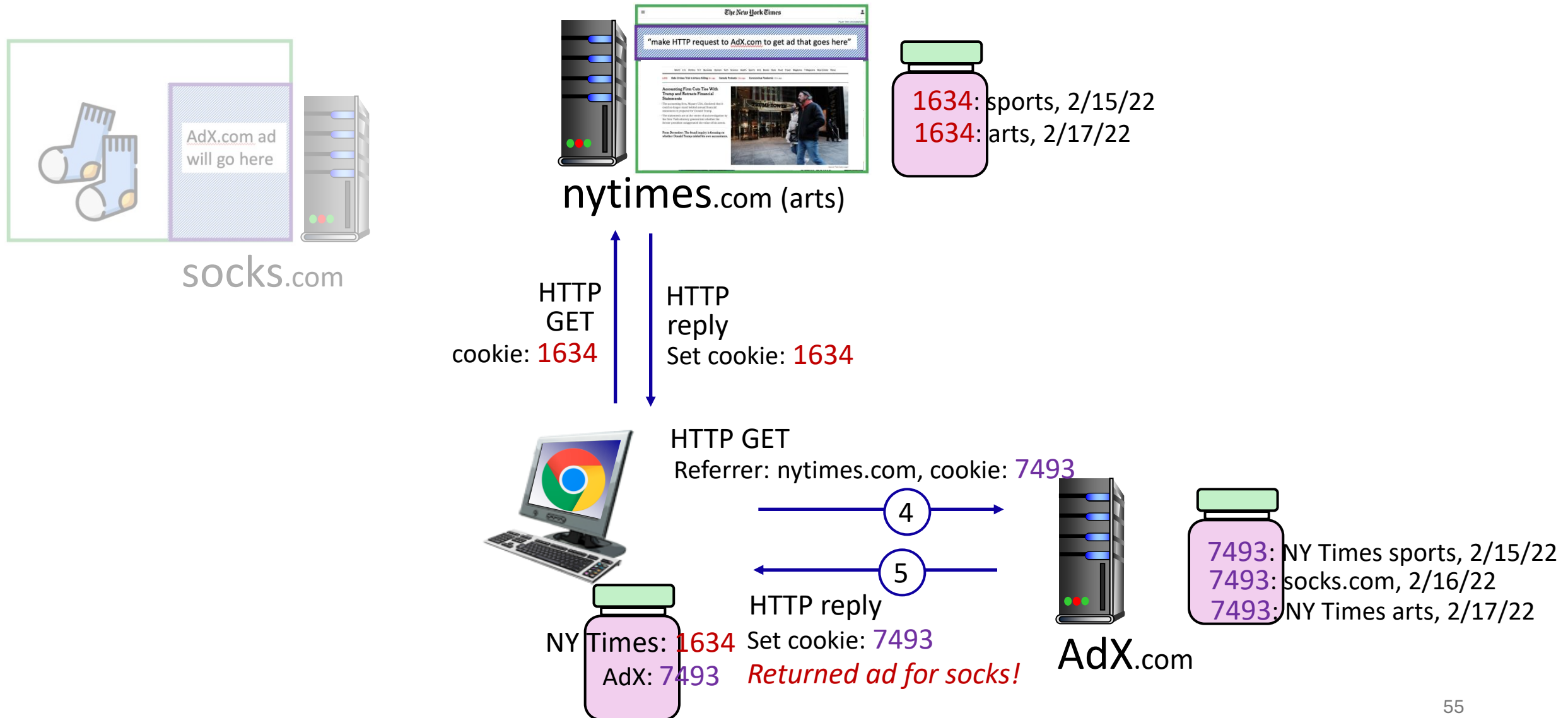
Cookies: tracking a user's browsing behavior



Cookies: tracking a user's browsing behavior



Cookies: tracking a user's browsing behavior (one day later)



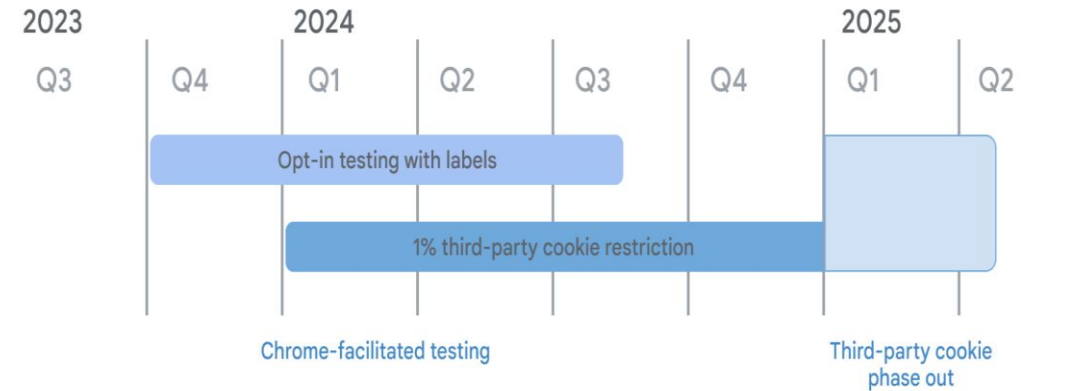
Cookies: tracking a user's browsing behavior

Cookies can be used to:

- track user behavior on a given website (**first party cookies**)
- track user behavior across multiple websites (**third party cookies**) without user ever choosing to visit tracker site (!)
- tracking may be *invisible* to user:
 - rather than displayed ad triggering HTTP GET to tracker, could be an invisible link

third party tracking via cookies:

- disabled by default in Firefox, Safari browsers
- ~~to be disabled in Chrome browser in 2023 which google backtrack in Oct 2024~~
- In July and April 2025 announcements, Google confirmed it would no longer pursue a complete, automatic phase-out of third-party cookies for all users.



GDPR (EU General Data Protection Regulation) and cookies

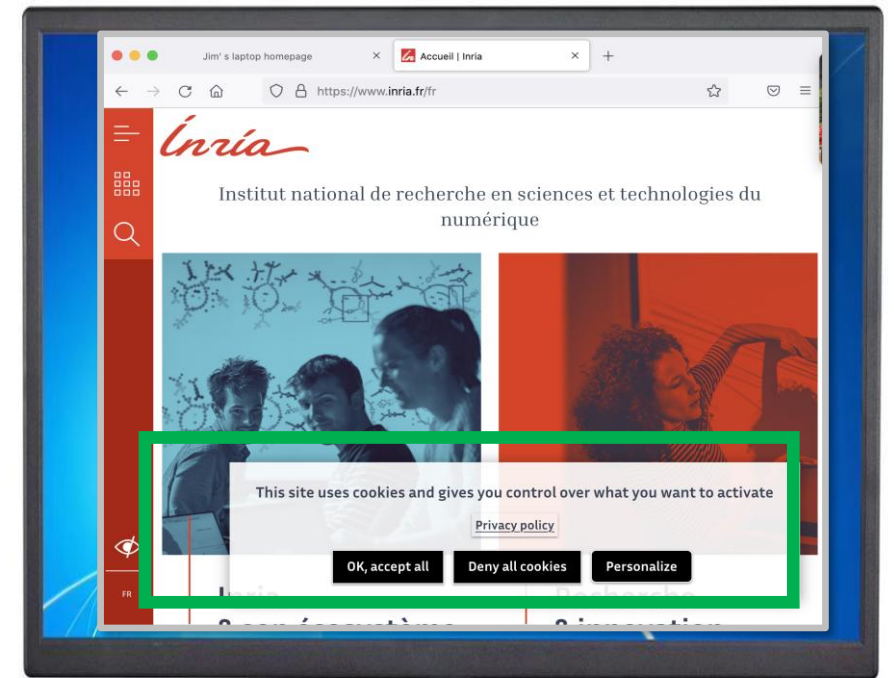
“Natural persons may be associated with online identifiers [...] such as internet protocol addresses, cookie identifiers or other identifiers [...].

This may leave traces which, in particular when combined with unique identifiers and other information received by the servers, may be used to create profiles of the natural persons and identify them.”

GDPR, recital 30 (May 2018)



when cookies can identify an individual, cookies are considered personal data, subject to GDPR personal data regulations



User has explicit control over whether or not cookies are allowed

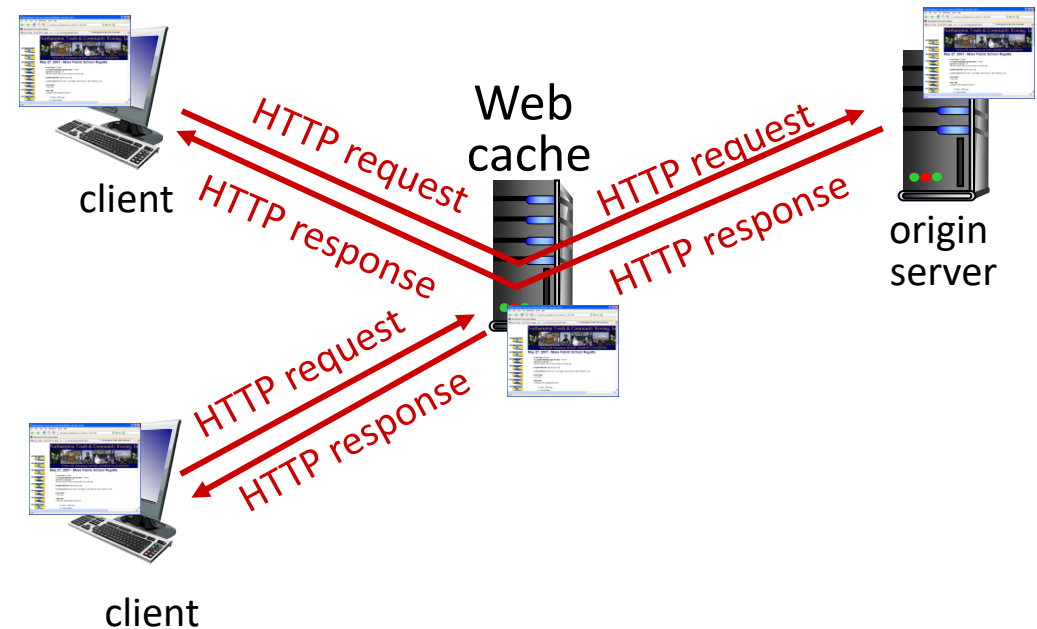


Web Cache & Browser Cache

Web caches

Goal: satisfy client requests without involving origin server

- user configures browser to point to a (local) *Web cache*
- browser sends all HTTP requests to cache
 - *if* object in cache: cache returns object to client
 - *else* cache requests object from origin server, caches received object, then returns object to client



Web caches (aka proxy servers)

- Web cache acts as both client and server
 - server for original requesting client
 - client to origin server
- server tells cache about object's allowable caching in response header:

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

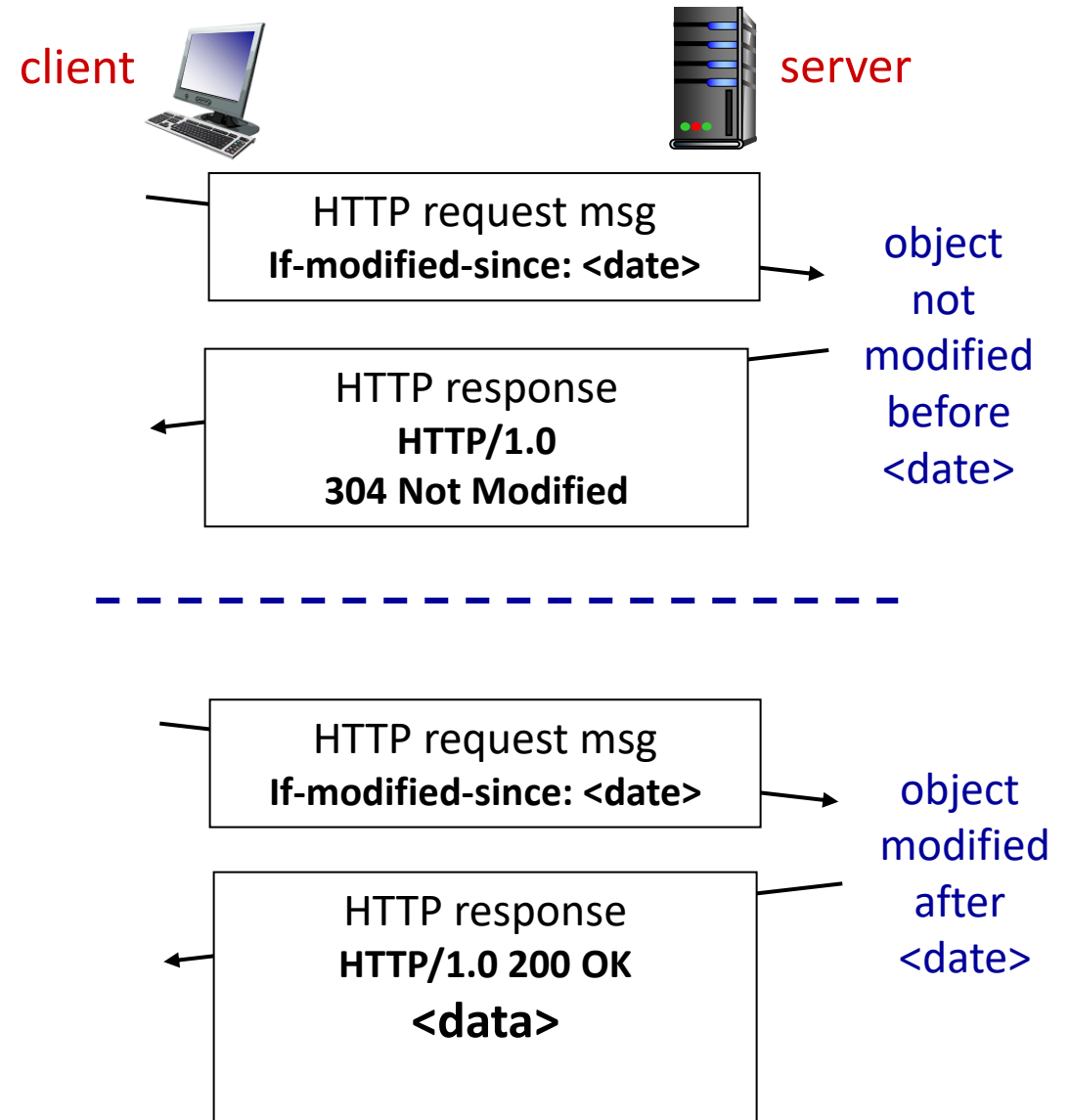
Why Web caching?

- reduce response time for client request
 - cache is closer to client
- reduce traffic on an institution's access link
- Internet is dense with caches
 - enables “poor” content providers to more effectively deliver content

Browser caching: Conditional GET

Goal: don't send object if browser has up-to-date cached version

- no object transmission delay (or use of network resources)
- **client:** specify date of browser-cached copy in HTTP request
If-modified-since: <date>
- **server:** response contains no object if browser-cached copy is up-to-date:
HTTP/1.0 304 Not Modified



HTTP/2

Key goal: decreased delay in multi-object HTTP requests

HTTP1.1: introduced multiple, pipelined GETs over single TCP connection

- server responds *in-order* (FCFS: first-come-first-served scheduling) to GET requests
- with FCFS, small object may have to wait for transmission (**head-of-line (HOL) blocking**) behind large object(s)
- loss recovery (retransmitting lost TCP segments) stalls object transmission

HTTP/2

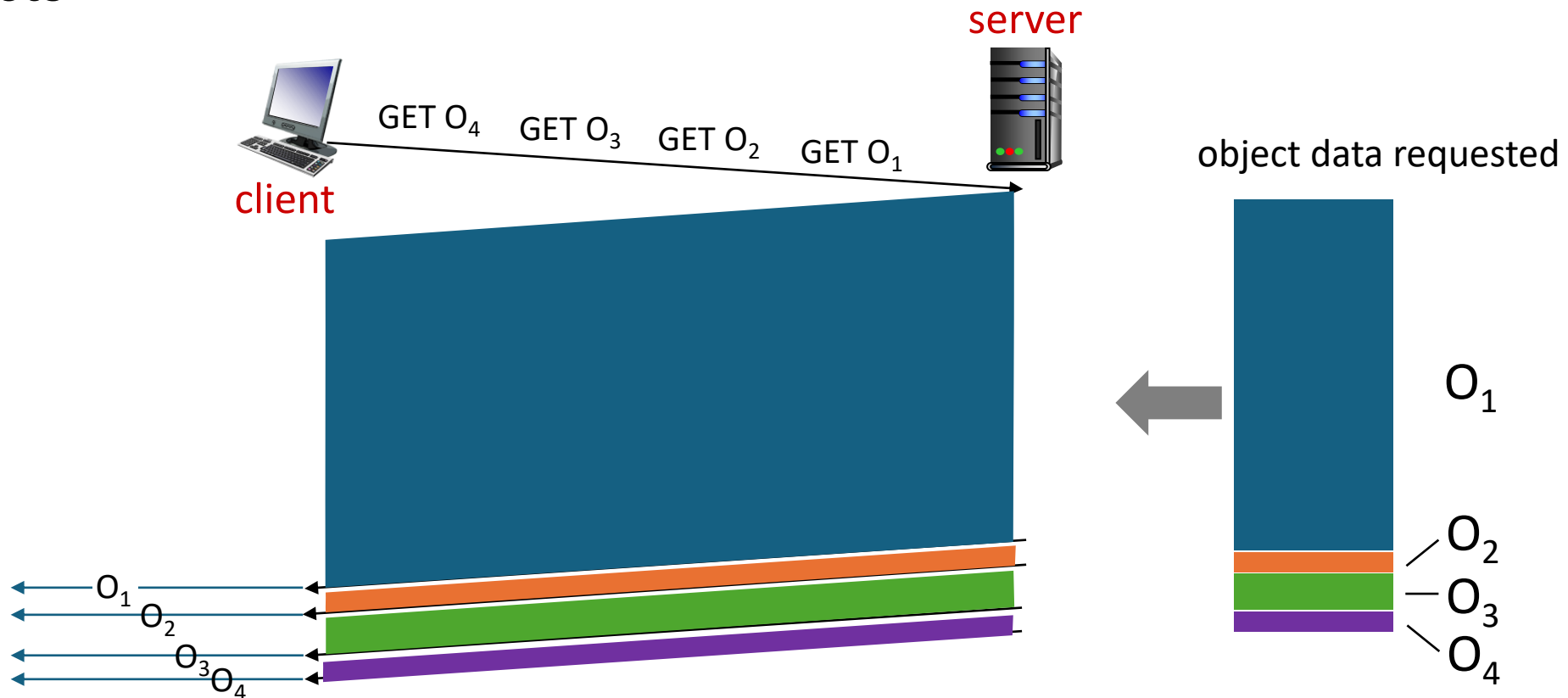
Key goal: decreased delay in multi-object HTTP requests

HTTP/2: [RFC 7540, 2015] increased flexibility at *server* in sending objects to client:

- methods, status codes, most header fields unchanged from HTTP 1.1
- transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- *push* unrequested objects to client
- divide objects into frames, schedule frames to mitigate HOL blocking

HTTP/2: mitigating HOL blocking

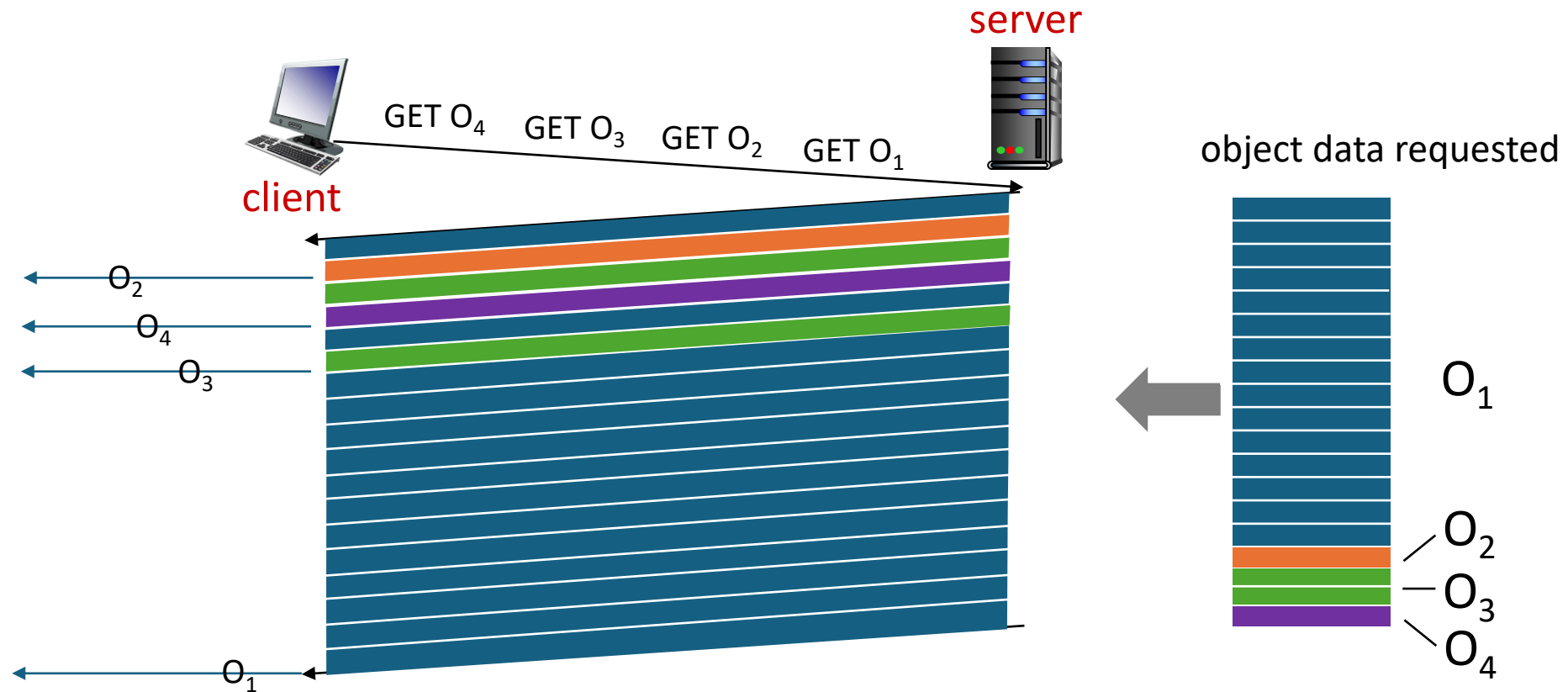
HTTP 1.1: client requests 1 large object (e.g., video file) and 3 smaller objects



objects delivered in order requested: O_2 , O_3 , O_4 wait behind O_1

HTTP/2: mitigating HOL blocking

HTTP/2: objects divided into frames, frame transmission interleaved



O₂, O₃, O₄ delivered quickly, O₁ slightly delayed

HTTP/2 to HTTP/3

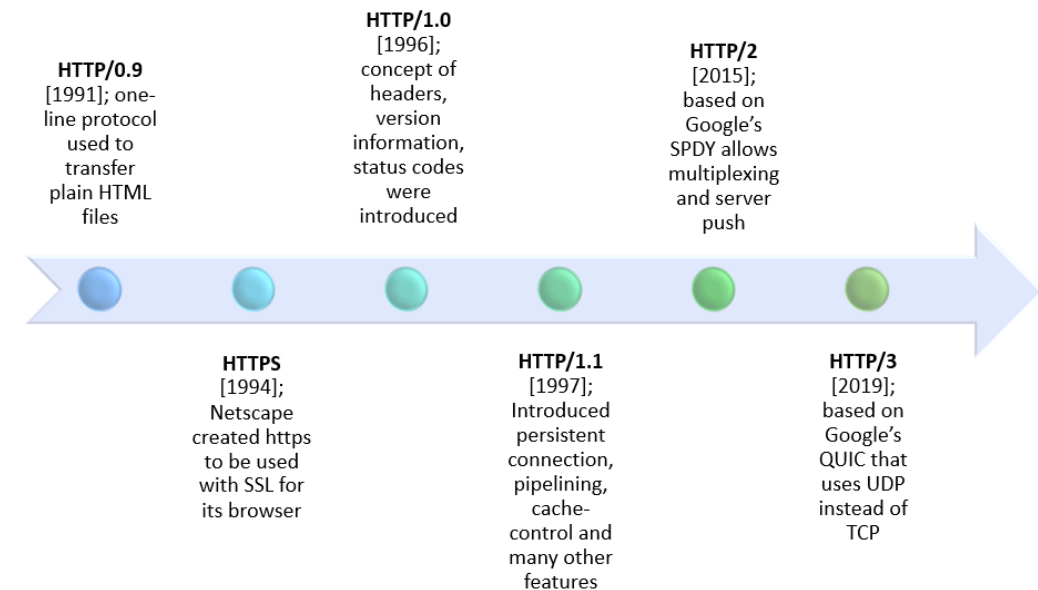
HTTP/2 over single TCP connection means:

- recovery from packet loss still stalls all object transmissions
 - as in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
- no security over vanilla TCP connection
- **HTTP/3**: adds security, per object error- and congestion-control (more pipelining) over UDP
 - more on HTTP/3 in transport layer

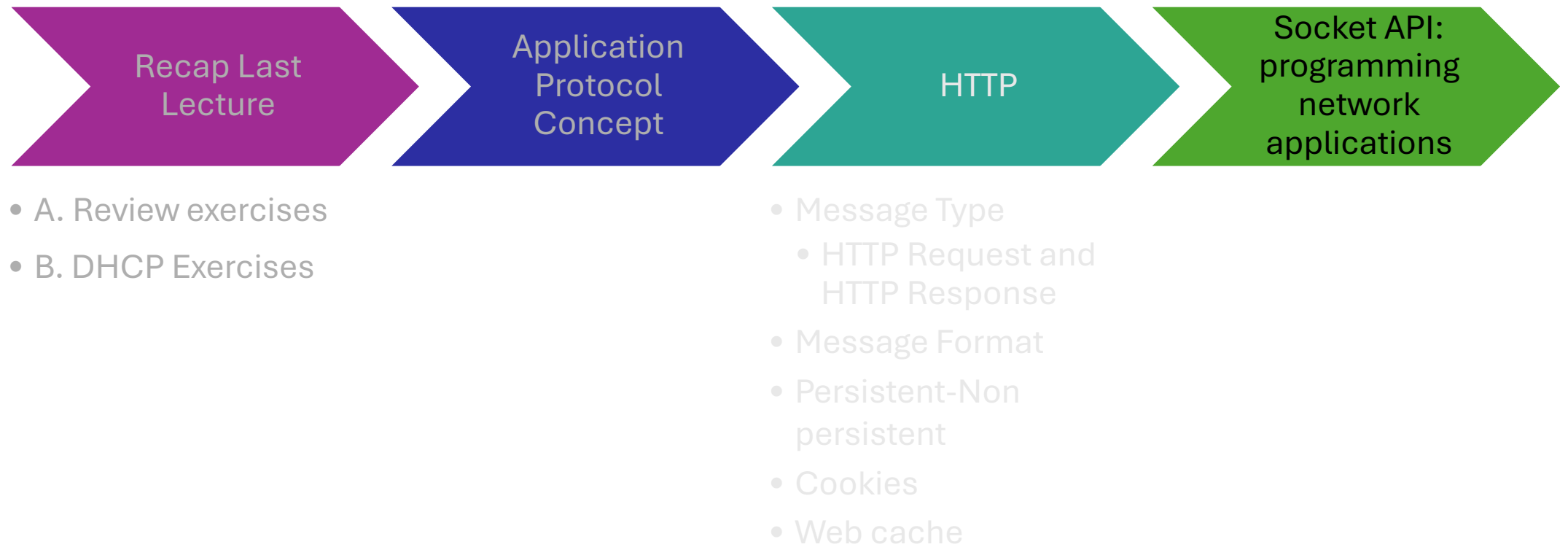
HTTP Versions

67

Comparison of protocol stack changes delivered with each new version after HTTP/1.0		
HTTP/1.1	HTTP/2	HTTP/3
- Some methods and response codes are added. "Keep-Alive" becomes officially supported. "Host" header becomes supported for Virtual Domain. - Syntax and semantics are separated.	- Support of parallel request transmission by "stream" (elimination of <i>HTTP</i> HoL Blocking). - Addition of flow-control and prioritization function in units of "stream". - Addition of server-push function (send related file without request.)	- Lower protocol changes from TCP+TLS to UDP+QUIC - Streams and flow-control function are moved to QUIC. - Parallel request transmission is supported by QUIC stream (eliminating <i>TCP</i> HoL Blocking).



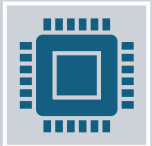
Outline



Socket programming with UDP and TCP



Slide page



นำมาจาก **Powerpoint slides**
Chapter 2: Application Layer

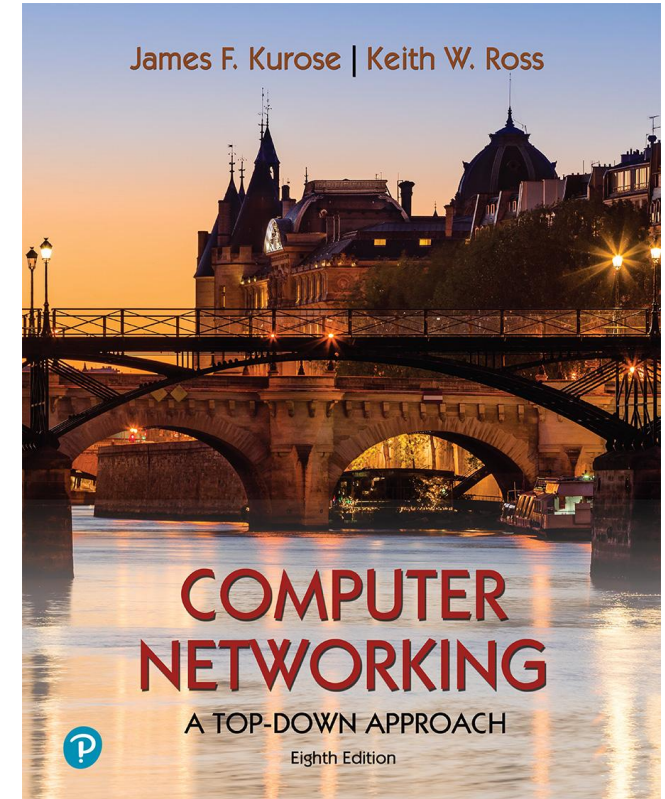
Page 2-106 – 2-115

•socket programming with UDP and TCP



Book : Computer Networking: A Top-Down Approach 8th edition
โดย **Jim Kurose, Keith Ross,**
Pearson, 2020

https://gaia.cs.umass.edu/kurose_ross/ppt.php



Computer Networking: A Top-Down Approach

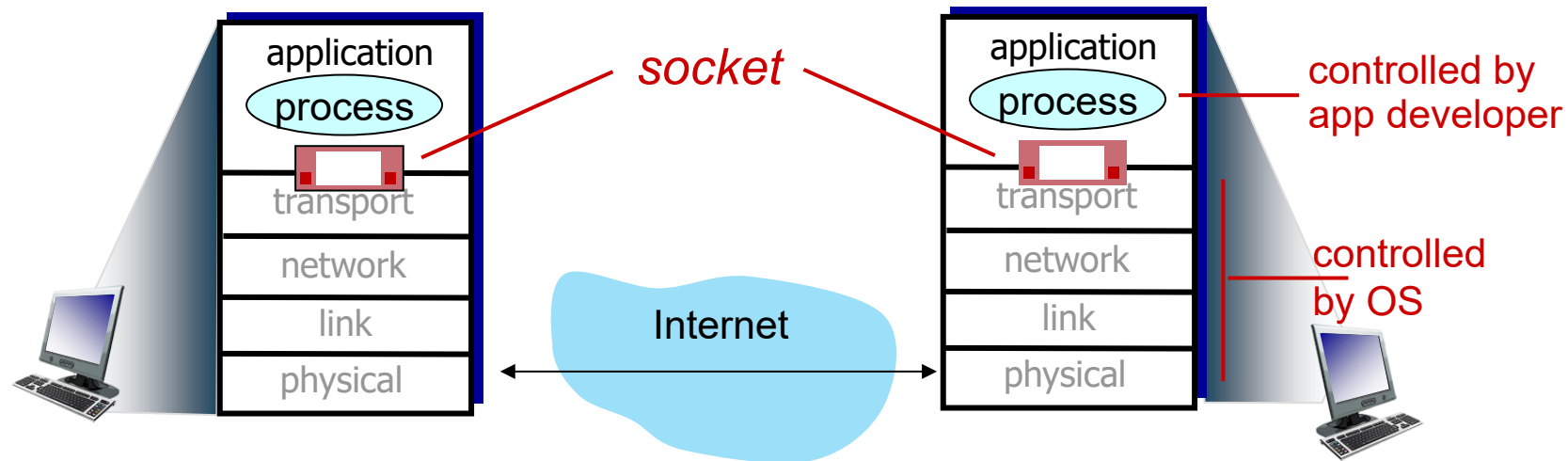
8th edition

Jim Kurose, Keith Ross
Pearson, 2020

Socket programming

goal: learn how to build client/server applications that communicate using sockets

socket: door between application process and end-end-transport protocol



Socket programming

Two socket types for two transport services:

- *UDP*: unreliable datagram
- *TCP*: reliable, byte stream-oriented

Application Example:

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

Socket programming with UDP

UDP: no “connection” between client and server:

- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides *unreliable* transfer of groups of bytes (“datagrams”) between client and server processes

Client/server socket interaction: UDP



server (running on serverIP)

create socket, port= x:
serverSocket =
socket(AF_INET,SOCK_DGRAM)

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

client



create socket:
clientSocket =
socket(AF_INET,SOCK_DGRAM)

Create datagram with serverIP address
And port=x; send datagram via
clientSocket

read datagram from
clientSocket

close
clientSocket

Example app: UDP client

Python UDPClient

```
include Python's socket library → from socket import *
                                   serverName = 'hostname'
                                   serverPort = 12000
create UDP socket → clientSocket = socket(AF_INET,
                                           SOCK_DGRAM)
get user keyboard input → message = input('Input lowercase sentence:')
attach server name, port to message; send into socket → clientSocket.sendto(message.encode(),
                                                                    (serverName, serverPort))
read reply data (bytes) from socket → modifiedMessage, serverAddress =
                                                                    clientSocket.recvfrom(2048)
print out received string and close socket → print(modifiedMessage.decode())
                                           clientSocket.close()
```

Example app: UDP server

Python UDPServer

```
from socket import *
serverPort = 12000
create UDP socket → serverSocket = socket(AF_INET, SOCK_DGRAM)
bind socket to local port number 12000 → serverSocket.bind(('', serverPort))
print('The server is ready to receive')
loop forever → while True:
    Read from UDP socket into message, getting → message, clientAddress = serverSocket.recvfrom(2048)
    client's address (client IP and port)      modifiedMessage = message.decode().upper()
    send upper case string back to this client → serverSocket.sendto(modifiedMessage.encode(),
                                                                    clientAddress)
```

Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

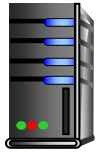
- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket*: client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
 - allows server to talk with multiple clients
 - client source port # and IP address used to distinguish clients (more in Chap 3)

Application viewpoint

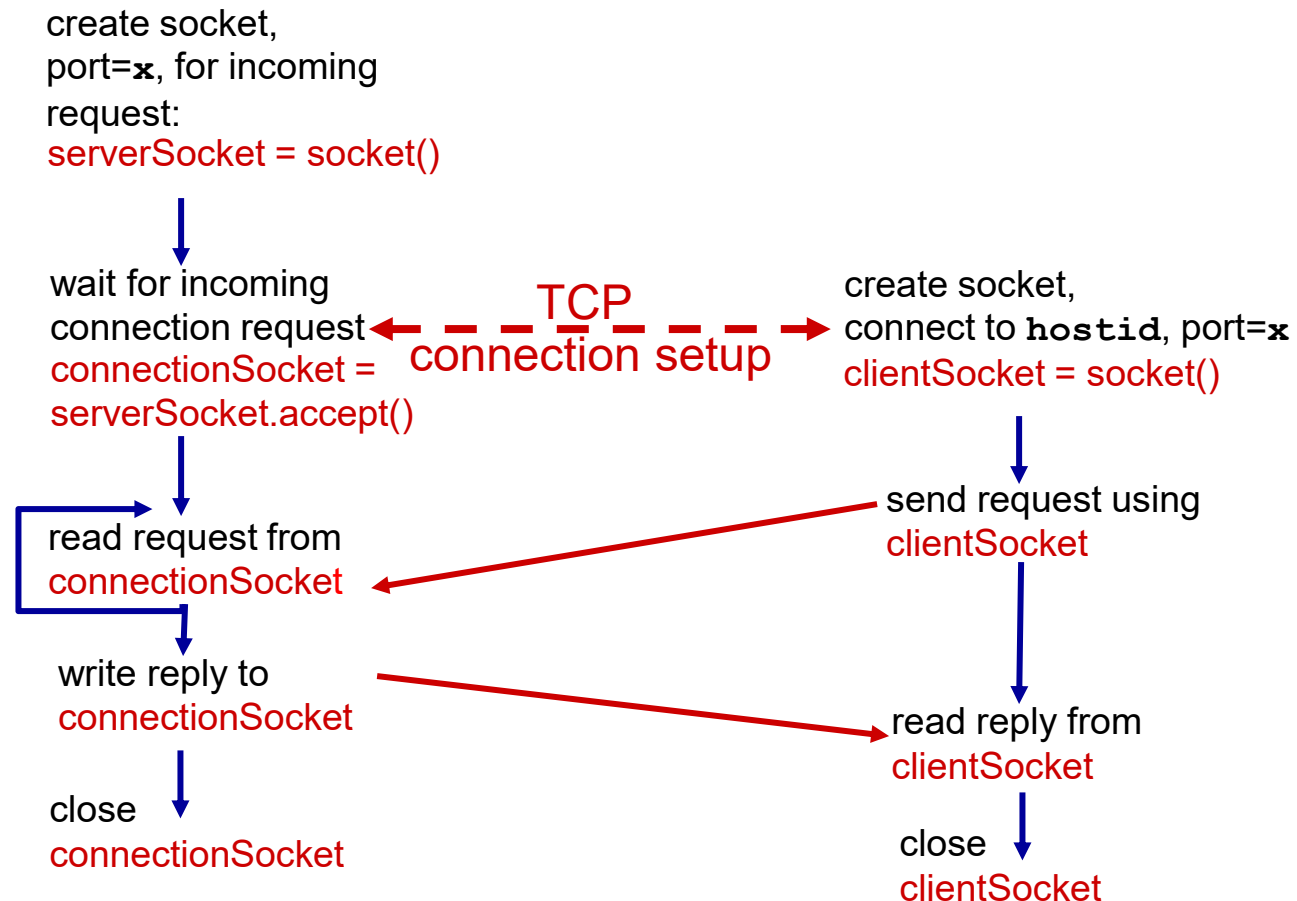
TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server processes

Client/server socket interaction: TCP



server (running on `hostid`)

client



Example app: TCP client

Python TCPClient

create TCP socket for server,
remote port 12000

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

No need to attach server name, port

Example app: TCP server

Python TCPServer

	from socket import *
	serverPort = 12000
create TCP welcoming socket →	serverSocket = socket(AF_INET,SOCK_STREAM)
	serverSocket.bind(('',serverPort))
server begins listening for incoming TCP requests →	serverSocket.listen(1)
	print('The server is ready to receive')
loop forever →	while True:
server waits on accept() for incoming requests, new socket created on return →	connectionSocket, addr = serverSocket.accept()
read bytes from socket (but not address as in UDP) →	sentence = connectionSocket.recv(1024).decode()
	capitalizedSentence = sentence.upper()
	connectionSocket.send(capitalizedSentence.encode())
close connection to this client (but <i>not</i> welcoming socket) →	connectionSocket.close()

Summary



Summary

our study of network application layer is now complete!

- application architectures
 - client-server
 - P2P
- application service requirements:
 - reliability, bandwidth, delay
- Internet transport service model
 - connection-oriented, reliable: TCP
 - unreliable, datagrams: UDP
- specific protocols:
 - HTTP
 - SMTP, IMAP
 - DNS
 - P2P: BitTorrent
- video streaming, CDNs
- socket programming:
TCP, UDP sockets

Chapter 2: Summary

Most importantly: learned about *protocols*!

- typical request/reply message exchange:
 - client requests info or service
 - server responds with data, status code
- message formats:
 - *headers*: fields giving info about data
 - *data*: info(payload) being communicated

important themes:

- centralized vs. decentralized
- stateless vs. stateful
- scalability
- reliable vs. unreliable message transfer
- “complexity at network edge”